Distributed Deep Neural Networks over the Cloud, the Edge and End Devices

Surat Teerapittayanon
Harvard University
Cambridge, MA, USA
Email: steerapi@seas.harvard.edu

Bradley McDanel
Harvard University
Cambridge, MA, USA
Email: mcdanel@fas.harvard.edu

H.T. Kung
Harvard University
Cambridge, MA, USA
Email: kung@harvard.edu

***Abstract*—We propose distributed deep neural networks (DDNNs) over distributed computing hierarchies, consisting of the cloud, the edge (fog) and end devices. While being able to accommodate inference of a deep neural network (DNN) in the cloud, a DDNN also allows fast and localized inference using shallow portions of the neural network at the edge and end devices. When supported by a scalable distributed computing hierarchy, a DDNN can scale up in neural network size and scale out in geographical span. Due to its distributed nature, DDNNs enhance sensor fusion, system fault tolerance and data privacy for DNN applications. In implementing a DDNN, we map sections of a DNN onto a distributed computing hierarchy. By jointly training these sections, we minimize communication and resource usage for devices and maximize usefulness of extracted features which are utilized in the cloud. The resulting system has built-in support for automatic sensor fusion and fault tolerance. As a proof of concept, we show a DDNN can exploit geographical diversity of sensors to improve object recognition accuracy and reduce communication cost. In our experiment, compared with the traditional method of offloading raw sensor data to be processed in the cloud, DDNN locally processes most sensor data on end devices while achieving high accuracy and is able to reduce the communication cost by a factor of over 20x.**

***Keywords*-distributed deep neural networks; deep neural networks; dnn; ddnn; embedded dnn; sensor fusion; distributed computing hierarchies; edge computing; cloud computing**

I. Introduction

Neural networks (NNs), and deep neural networks (DNNs) in particular, have achieved great success in numerous applications in recent years. For example, deep Convolutional Neural Networks (CNNs) continuously achieve state-of-the-art performances on various tasks in computer vision as shown in Figure 1. At the same time, the number of end devices, including Internet of Things (IoT) devices, has increased dramatically. These devices are appealing targets for machine learning applications as they are often directly connected to sensors (*e.g.*, cameras, microphones, gyroscopes) that capture a large quantity of input data in a streaming fashion.

However, the current state of machine learning systems on end devices leaves an unsatisfactory choice: either (1) offload input sensor data to large NN models (*e.g.*, DNNs) in the cloud, with the associated communication costs, latency issues and privacy concerns, or (2) perform classification directly on the end device using simple Machine Learning (ML) models *e.g.*, linear Support Vector Machine (SVM), leading to reduced system accuracy.

To address these shortcomings, it is natural to consider the use of a distributed computing approach. Hierarchically distributed computing structures consisting of the cloud, the edge and devices (see, *e.g.*, [1], [2]) have inherent advantages, such as supporting coordinated central and local decisions, and providing system scalability, for large-scale intelligent tasks based on geographically distributed IoT devices.

An example of one such distributed approach is to combine a small NN[1] model (less number of parameters) on end devices and a larger NN model (more number of parameters) in the cloud. The small model at an end device can quickly perform initial feature extraction, and also classification if the model is confident. Otherwise, the end device can fall back to the large NN model in the cloud, which performs further processing and final classification. This approach has the benefit of low communication costs compared to always offloading NN input to the cloud and can achieve higher accuracy compared to a simple model on device. Additionally, since a summary based on extracted features from the end device model are sent instead of raw sensor data, the system could provide better privacy protection.

However, this kind of distributed approach over a computing hierarchy is challenging for a number of reasons, including:

- End devices such as embedded sensor nodes often have limited memory and battery budgets. This makes it an issue to fit models on the devices that meet the required accuracy and energy constraints.
- A straightforward partitioning of NN models over a computing hierarchy may incur prohibitively large communication costs in transferring intermediate results between computation nodes.

[1]The term network layer may refer to either a layer in a NN or a layer in the distributed computing hierarchy (*e.g.*, edge or cloud). In order to remove ambiguity, when we refer to network layers for NN we explicitly use the term NN layers.

云、边缘和终端设备上的分布式深度神经网络

苏拉特·提拉皮塔亚农 (Surat Teerapittayanon)
Harvard University
Cambridge, MA, USA
Email: steerapi@seas.harvard.edu

布拉德利·麦克丹尼尔
Harvard University
Cambridge, MA, USA
Email: mcdanel@fas.harvard.edu

高温功夫
Harvard University
Cambridge, MA, USA
Email: kung@harvard.edu

***Abstract*-**我们提出基于分布式计算层次结构的分布式深度神经网络（DDNN），由云、边缘（雾）和终端设备组成。虽然能够适应云中深度神经网络 (DNN) 的推理，DDNN 还允许在边缘和终端设备上使用神经网络的浅层部分进行快速和局部推理。当得到可扩展的分布式计算层次结构的支持时，DDNN 可以扩大神经网络规模并扩大地理范围。由于其分布式特性，DDNN 增强了 DNN 应用的传感器融合、系统容错和数据隐私。在实现 DDNN 时，我们将 DNN 的各个部分映射到分布式计算层次结构上。通过联合训练这些部分，我们最大限度地减少了设备的通信和资源使用，并最大限度地提高了在云中使用的提取特征的有用性。由此产生的系统内置了对自动传感器融合和容错的支持。作为概念证明，我们展示了 DDNN 可以利用传感器的地理多样性来提高对象识别准确性并降低通信成本。在我们的实验中，与将原始传感器数据转移到云端进行处理的传统方法相比，DDNN 在终端设备上本地处理大部分传感器数据，同时实现高精度，并且能够将通信成本降低 20 倍以上。

***Keywords*-**分布式深度神经网络；深度神经网络；dnn；ddnn；嵌入式深度神经网络；传感器融合；分布式计算层次结构；边缘计算；云计算

一、简介

神经网络（NN），特别是深度神经网络（DNN）近年来在众多应用中取得了巨大成功。例如，深度卷积神经网络 (CNN) 在计算机视觉的各种任务上不断实现最先进的性能，如图 1 所示。与此同时，包括物联网 (IoT) 设备在内的终端设备的数量急剧增加。这些设备是机器学习应用程序的有吸引力的目标，因为它们通常直接连接到以流方式捕获大量输入数据的传感器（*e.g.*、摄像头、麦克风、陀螺仪）。

然而，终端设备上机器学习系统的当前状态留下了一个令人不满意的选择：(1) 将输入传感器数据卸载到云中的大型神经网络模型（*e.g.*，DNN），并产生相关的通信成本、延迟问题和隐私问题，或 (2) 使用简单的机器学习 (ML) 模型 *e.g.*、线性支持向量机 (SVM) 直接在终端设备上执行分类，从而导致系统精度降低。

为了解决这些缺点，很自然地要考虑使用分布式计算方法。由云、边缘和设备组成的分层分布式计算结构（参见*e.g.*，[1]，[2]）具有固有的优势，例如支持中央和本地协调决策，并为基于地理分布的物联网设备的大规模智能任务提供系统可扩展性。

此类分布式方法的一个示例是将终端设备上的小型 NN[1] 模型（参数数量较少）与云中的较大 NN 模型（参数数量较多）相结合。终端设备上的小模型可以快速执行初始特征提取，如果模型可信，还可以进行分类。否则，终端设备可以退回到云中的大型神经网络模型，该模型执行进一步的处理和最终分类。与始终将神经网络输入卸载到云端相比，这种方法的优点是通信成本低，并且与设备上的简单模型相比可以实现更高的准确性。此外，由于发送的是基于从终端设备模型中提取的特征的摘要，而不是原始传感器数据，因此系统可以提供更好的隐私保护。

然而，由于多种原因，这种基于计算层次结构的分布式方法具有挑战性，其中包括：

- 嵌入式传感器节点等终端设备的内存和电池预算通常有限。这使得在满足所需精度和能量约束的设备上拟合模型成为一个问题。
- 在计算层次上直接划分神经网络模型可能会在计算节点之间传输中间结果时产生巨大的通信成本。

[1]The term network layer may refer to either a layer in a NN or a layer in the distributed computing hierarchy (*e.g.*, edge or cloud). In order to remove ambiguity, when we refer to network layers for NN we explicitly use the term NN layers.

- Incorporating geographically distributed end devices is generally beyond the scope of DNN literature. When multiple sensor inputs on different end devices are used, they need to be aggregated together for a single classification objective. A trained NN will need to support such sensor fusion.
- Multiple models at the cloud, the edge and the device need to be learned jointly to allow coordinated decision making. Computation already performed on end device models should be useful for further processing on edge or cloud models.
- Usual layer-by-layer processing of a DNN from the NN's input layer all the way to the NN's output layer does not directly provide a mechanism for local and fast inference at earlier points in the neural networks (e.g., end devices).
- A balance is needed between the accuracy of a model (with the associated model size) at a given distributed computing layer and the cost of communicating to the layer above it. The solution must have reasonably good lower NN layers on the end devices capable of accurate local classification for some input while also providing useful features for classification in the cloud for other input.

To address these concerns under the same optimization framework, it is desirable that a system could train a *single* end-to-end model, such as a DNN, and partition it between end devices and the cloud[2], in order to provide a simpler and more principled approach.

To this end, we propose distributed deep neural networks (DDNNs) over distributed computing hierarchies, consisting of the cloud, the edge (fog) and geographically distributed end devices. In implementing a DDNN, we map sections of a single DNN onto a distributed computing hierarchy. By jointly training these sections, we show that DDNNs can effectively address the aforementioned challenges. Specifically, while being able to accommodate inference of a DNN in the cloud, a DDNN allows fast and localized inference using some shallow portions of the DNN at the edge and end devices. Moreover, via distributed computing, DDNNs naturally enhance sensor fusion, data privacy and system fault tolerance for DNN applications. When supported by a scalable distributed computing hierarchy, a DDNN can scale up in neural network size and scale out in geographical span.

DDNN leverages our earlier work on BranchyNet [3] which allows early exit points to be placed in a DNN. Samples can be classified and exited locally when the system is confident and offloaded to the edge and the cloud when additional processing is required. In addition, DDNN leverages the recent work of binary neural networks (BNNs) [4], which

[2]For presentation simplicity, we often just consider the device-cloud scenario. Our methodology can similarly apply to general device-edge (fog)-cloud scenarios.

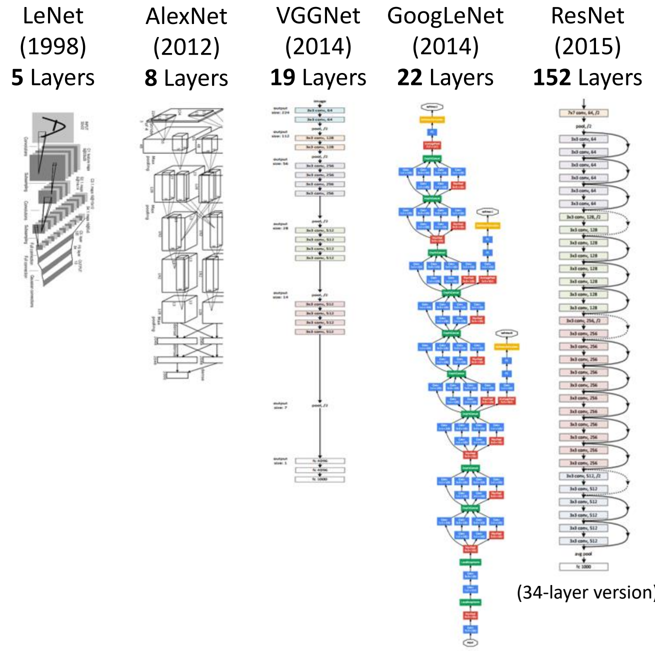

Figure 1. Progression towards deeper neural network structures in recent years (see, *e.g.*, [6], [7], [8], [9], [10]).

greatly reduce the required memory cost of neural network layers and enables multi-layer NNs to run on end devices with small memory footprints [5]. By training DDNN end-to-end, the network optimally configures lower NN layers to support local inference at end devices, and higher NN layers in the cloud to improve overall classification accuracy of the system. As a proof of concept, we show a DDNN can exploit geographical diversity of sensors (on a multi-view multi-camera dataset) in sensor fusion to improve recognition accuracy.

The contributions of this paper include

1) A novel DDNN framework and its implementation that maps sections of a DNN onto a distributed computing hierarchy.
2) A joint training method that minimizes communication and resource usage for devices and maximizes usefulness of extracted features which are utilized in the cloud, while allowing low-latency classification via early exit for a high percentage of input samples.
3) Aggregation schemes that allows automatic sensor fusion of multiple sensor inputs to improve the overall performance (accuracy and fault tolerance) of the system.

The DDNN codebase is open source and can be found here: https://github.com/kunglab/ddnn.

II. Related Work

In this section, we briefly review related work in distributed computing hierarchies and recent deep learning

- 整合地理上分布的终端设备通常超出了 DNN 文献的范围。当不同终端设备上使用多个传感器输入时，需要将它们聚合在一起以实现单个分类目标。经过训练的神经网络需要支持这种传感器融合。

- 需要共同学习云端、边缘和设备的多个模型，以实现协调决策。已在终端设备模型上执行的计算对于在边缘或云模型上进行进一步处理应该很有用。

- 通常，DNN 从神经网络的输入层一直到神经网络的输出层的逐层处理并不能直接提供神经网络早期点（例如终端设备）的本地快速推理机制。

- 给定分布式计算层的模型精度（以及相关模型大小）和与其上方层通信的成本之间需要取得平衡。该解决方案必须在终端设备上具有相当好的较低神经网络层，能够对某些输入进行准确的本地分类，同时还为其他输入在云中进行分类提供有用的功能。

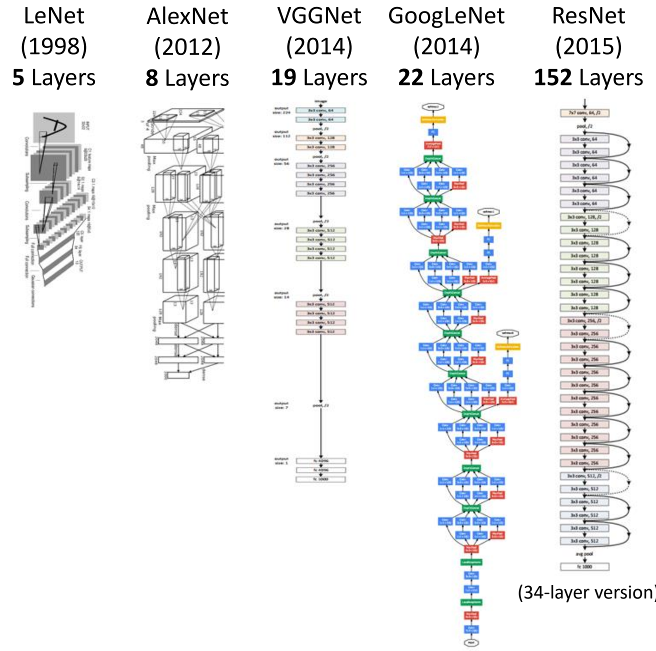

图 1. 近年来更深层次神经网络结构的进展（参见 *e.g.*、[6]、[7]、[8]、[9]、[10]）。

为了在同一优化框架下解决这些问题，系统最好能够训练 *single* 端到端模型（例如 DNN），并将其在终端设备和云 [2] 之间进行划分，以便提供更简单、更有原则性的方法。

为此，我们提出了基于分布式计算层次结构的分布式深度神经网络（DDNN），其中包括云、边缘（雾）和地理上分布的终端设备。在实现 DDNN 时，我们将单个 DNN 的各个部分映射到分布式计算层次结构上。通过联合训练这些部分，我们表明 DDNN 可以有效解决上述挑战。具体来说，DDNN 在能够适应云中 DNN 推理的同时，还允许在边缘和终端设备上使用 DNN 的一些浅层部分进行快速和局部推理。此外，通过分布式计算，DDNN 自然地增强了 DNN 应用的传感器融合、数据隐私和系统容错能力。当得到可扩展的分布式计算层次结构的支持时，DDNN 可以扩大神经网络规模并扩大地理范围。

DDNN 利用了我们早期在 BranchyNet [3] 上的工作，它允许将早期退出点放置在 DNN 中。当系统有信心时，可以在本地对样本进行分类和退出，并在需要额外处理时将其卸载到边缘和云端。此外，DDNN 利用了二元神经网络 (BNN) [4] 的最新成果，大大降低了神经网络层所需的内存成本，并使多层神经网络能够在内存占用较小的终端设备上运行[5]。通过端到端训练 DDNN，网络优化配置较低的 NN 层以支持终端设备的本地推理，并在云中优化配置较高的 NN 层以提高系统的整体分类准确性。作为概念证明，我们展示了 DDNN 可以在传感器融合中利用传感器的地理多样性（在多视图多相机数据集上）来提高识别精度。

本文的贡献包括
1) 一种新颖的 DDNN 框架及其实现，将 DNN 的各个部分映射到分布式计算层次结构上。 2）一种联合训练方法，最大限度地减少设备的通信和资源使用，并最大限度地提高在云中使用的提取特征的有用性，同时允许通过提前退出高比例的输入样本来实现低延迟分类。 3）聚合方案，允许多个传感器输入的自动传感器融合，以提高系统的整体性能（准确性和容错性）。

DDNN 代码库是开源的，可以在此处找到：https://github.com/kunglab/ddnn。

二.相关工作

在本节中，我们简要回顾分布式计算层次结构和最近深度学习的相关工作

[2]For presentation simplicity, we often just consider the device-cloud scenario. Our methodology can similarly apply to general device-edge (fog)-cloud scenarios.

algorithms that enable our proposed method to run in a distributed fashion. We then discuss other approaches involving distributed deep networks.

A. Distributed Computing Hierarchy

The framework of a large-scale distributed computing hierarchy has assumed new significance in the emerging era of IoT. It is widely expected that most of data generated by the massive number of IoT devices must be processed locally at the devices or at the edge, for otherwise the total amount of sensor data for a centralized cloud would overwhelm the communication network bandwidth. In addition, a distributed computing hierarchy offers opportunities for system scalability, data security and privacy, as well as shorter response times (see, *e.g.*, [2], [11]). For example, in [11], a face recognition application shows a reduced response time is achieved when a smartphone's photos are proceeded by the edge (fog) as opposed to the cloud. In this paper, we show that DDNN can systematically exploit the inherent advantages of a distributed computing hierarchy for DNN applications and achieve similar benefits.

B. Deep Neural Network Extensions

Binarized neural networks (BNNs) are a recent type of neural networks, where the weights in linear and convolutional layers are constrained to $\{-1, 1\}$ (stored as 0 and 1 respectively). This representation has been shown to achieve similar classification accuracy for some datasets such as MNIST and CIFAR-10 [12] when compared to a standard floating-point neural network while using less memory and reduced computation due to the binary format [4]. Embedded binarized neural networks (eBNNs) extends BNNs to allow the network to fit on embedded devices by reducing floating-point temporaries through reordering the operations in inference [5]. These compact models are especially attractive in end device settings, where memory can be a limiting factor and low power consumption is required. In DDNN, we use BNNs, eBNNs and the alike to accommodate the end devices, so that they can be jointly trained with the NN layers in the edge and cloud.

BranchyNet proposed a solution of classifying samples at earlier points in a neural network, called early exit points, through the use of an entropy-based confidence criteria [3]. If at an early exit point a sample is deemed confident based on the entropy of the computed probability vector for target classes, then it is classified and no further computation is performed by the higher NN layers. In DDNN, exit points are placed at physical boundaries (*e.g.*, between the last NN layer on an end device and the first NN layer in the next higher layer of the distributed computing hierarchy such as the edge or the cloud). Input samples that can already be classified early will exit locally, thereby achieving a lowered response latency and saving communication to the next physical boundary. With similar objectives, SACT [13] allocates computation on a per region basis in an image, and exits each region independently when it is deemed to be of sufficient quality.

C. Distributed Training of Deep Networks

Current research on distributing deep networks is mainly focused on improving the runtime of training the neural network. In 2012, Dean *et al.* proposed DistBelief, which maps large DNNs over thousands of CPU cores during training [14]. More recently, several methods have been proposed to scale up DNN training across GPU clusters [15], [16], which further reduces the runtime of network training. Note that this form of distributing DNNs (over homogeneous computing units) is fundamentally different from the notion presented in this paper. We proposes a way to train and perform feedforward inference over deep networks that can be deployed over a distributed computing hierarchy, rather than processed in parallel over bus- or switch-connected CPUs or GPUs in the cloud.

III. Proposed Distributed Deep Neural Networks

In this section we give an overview of the proposed distributed deep neural network (DDNN) architecture and describe how training and inference in DDNN is performed.

A. DDNN Architecture

DDNN maps a trained DNN onto heterogeneous physical devices distributed locally, at the edge, and in the cloud. Since DDNN relies on a jointly trained DNN framework at all parts in the neural network, for both training and inference, many of the difficult engineering decisions are greatly simplified. Figure 2 provides an overview of the DDNN architecture. The configurations presented show how DDNN can scale the inference computation across different physical devices. The cloud-based DDNN in (a) can be viewed as the standard DNN running in the cloud as described in the introduction. In this case, sensor input captured on end devices is sent to the cloud in original format (raw input format), where all layers of DNN inference is performed.

We can extend this model to include a single end device, as shown in (b), by performing a portion of the DNN inference computation on the device rather than sending the raw input to the cloud. Using an exit point after device inference, we may classify those samples which the local network is confident about, without sending any information to the cloud. For more difficult cases, the intermediate DNN output (up to the local exit) is sent to the cloud, where further inference is performed using additional NN layers and a final classification decision is made. Note that the intermediate output can be designed to be much smaller than the sensor input (*e.g.*, a raw image from a video camera), and therefore drastically reduce the network communication required between the end device and the cloud. The details

使我们提出的方法能够以分布式方式运行的算法。然后我们讨论涉及分布式深度网络的其他方法。

A. Distributed Computing Hierarchy

大规模分布式计算层次结构的框架在新兴的物联网时代具有新的意义。人们普遍认为，大量物联网设备生成的大部分数据必须在设备本地或边缘进行处理，否则集中式云的传感器数据总量将压垮通信网络带宽。此外，分布式计算层次结构还为系统可扩展性、数据安全性和隐私性以及更短的响应时间提供了机会（参见*e.g.*、[2]、[11]）。例如，在[11]中，人脸识别应用程序显示，当智能手机的照片通过边缘（雾）而不是云端进行处理时，可以缩短响应时间。在本文中，我们表明 DDNN 可以系统地利用 DNN 应用的分布式计算层次结构的固有优势，并实现类似的好处。

B. Deep Neural Network Extensions

二值化神经网络（BNN）是一种最新类型的神经网络，其中线性层和卷积层中的权重被限制为 $\{-1, 1\}$ (分别存储为 0 和 1)。与标准浮点神经网络相比，这种表示形式已被证明可以对某些数据集（例如 MNIST 和 CIFAR-10 [12]）实现类似的分类精度，同时由于二进制格式而使用更少的内存和减少计算量 [4]。嵌入式二值化神经网络（eBNN）扩展了 BNN，通过重新排序推理操作来减少浮点临时数据，从而使网络能够适应嵌入式设备[5]。这些紧凑型型号在终端设备设置中尤其有吸引力，因为内存可能是限制因素并且需要低功耗。在DDNN中，我们使用BNN、eBNN等来容纳终端设备，以便它们可以与边缘和云端的NN层联合训练。

BranchyNet 提出了一种通过使用基于熵的置信度标准对神经网络中早期点（称为早期退出点）的样本进行分类的解决方案 [3]。如果在早期退出点，根据目标类的计算概率向量的熵，样本被认为是可信的，那么它就会被分类，并且更高的 NN 层不会执行进一步的计算。在 DDNN 中，出口点放置在物理边界（*e.g.*，终端设备上的最后一个 NN 层与分布式计算层次结构的下一个较高层（例如边缘或云）中的第一个 NN 层之间）。已经可以提前分类的输入样本将在本地退出，从而降低响应延迟并节省与下一个物理边界的通信。出于类似的目标，SACT [13] 在图像中按区域分配计算，并在认为质量足够时独立退出每个区域。

C. Distributed Training of Deep Networks

目前分布式深度网络的研究主要集中在提高神经网络训练的运行时间。 2012 年，Dean *et al.* 提出了 DistBelief，它在训练期间将大型 DNN 映射到数千个 CPU 核心上 [14]。最近，人们提出了几种跨 GPU 集群扩展 DNN 训练的方法 [15]、[16]，这进一步减少了网络训练的运行时间。请注意，这种形式的分布式 DNN（在同构计算单元上）与本文提出的概念根本不同。我们提出了一种在深度网络上训练和执行前馈推理的方法，该网络可以部署在分布式计算层次结构上，而不是通过云中总线或交换机连接的 CPU 或 GPU 并行处理。

三．提议的分布式深度神经网络

在本节中，我们概述了所提出的分布式深度神经网络 (DDNN) 架构，并描述了 DDNN 中的训练和推理是如何进行的。

A. DDNN Architecture

DDNN 将经过训练的 DNN 映射到分布在本地、边缘和云端的异构物理设备。由于 DDNN 在神经网络的所有部分都依赖于联合训练的 DNN 框架，无论是训练还是推理，许多困难的工程决策都得到了极大的简化。图 2 提供了 DDNN 架构的概述。所提供的配置展示了 DDNN 如何跨不同物理设备扩展推理计算。 (a) 中基于云的 DDNN 可以被视为在云中运行的标准 DNN，如简介中所述。在这种情况下，终端设备上捕获的传感器输入会以原始格式（原始输入格式）发送到云端，在云端执行 DNN 推理的所有层。

我们可以扩展此模型以包含单个终端设备，如 (b) 所示，方法是在设备上执行部分 DNN 推理计算，而不是将原始输入发送到云端。使用设备推理后的退出点，我们可以对本地网络有信心的那些样本进行分类，而无需向云端发送任何信息。对于更困难的情况，中间 DNN 输出（直到本地出口）被发送到云端，在云端使用额外的 NN 层进行进一步的推理，并做出最终的分类决策。请注意，中间输出可以设计为远小于传感器输入（*e.g.*，来自摄像机的原始图像），因此大大减少了终端设备和云之间所需的网络通信。详细内容

of how communication is considered in the network is discussed in section III-E.

DDNN can also be extended to multiple end devices which may be geographically distributed, shown in (c), that work together to make a classification decision. Here, each end device performs local computation as in (b), but their output is aggregated together before the local exit point. Since the entire DDNN is jointly trained across all end devices and exit points, the network automatically aggregates the input with the objective of achieving maximum classification accuracy. This automatic data fusion (sensor fusion) simplifies runtime inference by avoiding the necessity of manually combining output from multiple end devices. We will discuss the design of feature aggregation in detail in section III-B. As before, if the local exit point is not confident about the sample, each end devices sends intermediate output to the cloud, where another round of feature aggregation is performed before making a final classification decision.

DDNN scales vertically as well, by using an edge layer in the distributed computing hierarchy between the end devices and cloud, shown in (d) and (e). The edge acts similarly to the cloud, by taking output from the end devices, performing aggregation and classification if possible, and forwarding its own intermediate output to the cloud if more processing is needed. In this way, DDNN naturally adjusts the network communication and response time of the system on a per sample basis. Samples that can be correctly classified locally are exiting without any communication to the edge or cloud. Samples that require more feature extraction than can be provided locally are sent to the edge, and eventually the cloud if necessary. Finally, DDNNs can also scale geographically across the edge layer as well, which is shown in (f).

B. DDNN Aggregation Methods

In DDNN configurations with multiple end devices (*e.g.*, (c), (e), and (f) in Figure 2), the output from each end device must be aggregated in order to perform classification. We present several different schemes for aggregating the output. Each aggregation method makes different assumptions about how the device output should be combined and therefore can result in different system accuracy. We present three approaches:

- Max pooling (MP). MP aggregates the input vectors by taking the max of each component. Mathematically, max pooling can be written as

$$\hat{v}_j = \max_{1 \leq i \leq n} v_{ij},$$

where n is the number of inputs and v_{ij} is the j-th component of the input vector and $\hat{v}_j$ is the j-th component of the resulting output vector.

- Average pooling (AP). AP aggregates the input vectors by taking the average of each component. This is written as

$$\hat{v}_j = \sum_{i=1}^{n} \frac{v_{ij}}{n},$$

where n is the number of inputs and v_{ij} is the j-th component of the input vector and $\hat{v}_j$ is the j-th component of the resulting output vector. Averaging may reduce noisy input presented in some end devices.

- Concatenation (CC). CC simply concatenates the input vectors together. CC retains all information which is useful for higher layers (*e.g.*, the cloud) that can use the full information to extract higher level features. Note that this expands the dimension of the resulting vector. To map this vector back to the same number of dimensions as input vectors, we add an additional linear layer.

We analyzes these aggregation methods in Section IV-C.

C. DDNN Training

While DDNN inference is distributed over the distributed computing hierarchy, the DDNN system can be trained on a single powerful server or in the cloud. One aspect of DDNN that is different from most conventional DNN pipelines is the use of multiple exit points as shown in Figure 2. At training time, the loss from each exit is combined during backpropagation so that the entire network can be jointly trained, and each exit point achieves good accuracy relative to its depth. For this work, we follow joint training as described in GoogleNet [9] and BranchyNet [3].

For the system evaluation discussed in Section IV, we apply DDNNs to a classification task. We use the softmax cross entropy loss function as the optimization objective. We now describe formally how we train DDNNs. Let $\boldsymbol{y}$ be a one-hot ground-truth label vector, $\boldsymbol{x}$ be an input sample and $\mathcal{C}$ be the set of all possible labels. For each exit, the softmax cross entropy objective function can be written as

$$L(\hat{\boldsymbol{y}}, \boldsymbol{y}; \theta) = -\frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} y_c \log \hat{y_c},$$

where

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{z}) = \frac{\exp(\boldsymbol{z})}{\sum\limits_{c \in \mathcal{C}} \exp(z_c)},$$

and

$$\boldsymbol{z} = f_{\text{exit}_n}(\boldsymbol{x}; \theta),$$

where f_{exit_n} is a function representing the computation of the neural network layers from an entry point to the n-th exit branch and θ represents the network parameters such as weights and biases of those layers.

To train the DDNN we form a joint optimization problem as minimizing a weighted sum of the loss functions of each

III-E 节讨论了如何在网络中考虑通信。

DDNN 还可以扩展到地理上分布的多个终端设备，如 (c) 所示，它们协同工作以做出分类决策。这里，每个终端设备执行如(b)中所示的本地计算，但它们的输出在本地出口点之前聚合在一起。由于整个 DDNN 是在所有终端设备和出口点上联合训练的，因此网络会自动聚合输入，以实现最大分类精度。这种自动数据融合（传感器融合）避免了手动组合多个终端设备输出的必要性，从而简化了运行时推理。我们将在第 III-B 节中详细讨论特征聚合的设计。和以前一样，如果本地出口点对样本不放心，每个终端设备都会将中间输出发送到云端，在云端执行另一轮特征聚合，然后再做出最终的分类决策。

DDNN 还可以通过使用终端设备和云之间的分布式计算层次结构中的边缘层来垂直扩展，如 (d) 和 (e) 所示。边缘的行为与云类似，从终端设备获取输出，如果可能的话执行聚合和分类，如果需要更多处理，则将其自己的中间输出转发到云。这样，DDNN 自然会根据每个样本调整系统的网络通信和响应时间。可以在本地正确分类的样本无需与边缘或云进行任何通信即可退出。需要比本地提供更多特征提取的样本被发送到边缘，并在必要时最终发送到云端。最后，DDNN 还可以跨边缘层进行地理扩展，如 (f) 所示。

B. DDNN Aggregation Methods

在具有多个终端设备的 DDNN 配置中（图 2 中的 *e.g.*、(c)、(e) 和 (f)），必须聚合每个终端设备的输出才能执行分类。我们提出了几种不同的聚合输出方案。每种聚合方法对如何组合设备输出做出不同的假设，因此可能导致不同的系统精度。我们提出三种方法:

- 最大池化 (MP)。 MP 通过取每个分量的最大值来聚合输入向量。从数学上讲，最大池化可以写为

$$\hat{v}_j = \max_{1\leq i\leq n} v_{ij},$$

  其中 n 是输入的数量，v_{ij} 是输入向量的第 j 个分量，$\hat{v}_j$ 是结果输出向量的第 j 个分量。

- 平均池化（AP）。 AP 通过取每个分量的平均值来聚合输入向量。这是写成

$$\hat{v}_j = \sum_{i=1}^{n} \frac{v_{ij}}{n},$$

  其中 n 是输入的数量，v_{ij} 是输入向量的第 j 个分量，$\hat{v}_j$ 是结果输出向量的第 j 个分量。平均可以减少某些终端设备中出现的噪声输入。

- 连接（CC）。 CC 只是将输入向量连接在一起。 CC 保留对高层（*e.g.*，云）有用的所有信息，高层可以使用完整信息来提取更高级别的特征。请注意，这会扩展所得向量的维度。为了将该向量映射回与输入向量相同的维度，我们添加了一个额外的线性层。

我们在第 IV-C 节中分析了这些聚合方法。

C. DDNN Training

虽然 DDNN 推理分布在分布式计算层次结构上，但 DDNN 系统可以在单个强大的服务器或云中进行训练。 DDNN 与大多数传统 DNN 管道不同的一个方面是使用多个出口点，如图 2 所示。在训练时，每个出口的损失在反向传播期间合并，以便整个网络可以联合训练，并且每个出口点相对于其深度都获得良好的精度。对于这项工作，我们遵循 GoogleNet [9] 和 BranchyNet [3] 中所述的联合训练。

对于第四节中讨论的系统评估，我们将 DDNN 应用于分类任务。我们使用softmax交叉熵损失函数作为优化目标。我们现在正式描述如何训练 DDNN。令 $\boldsymbol{y}$ 为单热真实标签向量，$\boldsymbol{x}$ 为输入样本，$\mathcal{C}$ 为所有可能标签的集合。对于每个出口，softmax交叉熵目标函数可以写为

$$L(\hat{\boldsymbol{y}}, \boldsymbol{y}; \theta) = -\frac{1}{|\mathcal{C}|} \sum_{c\in\mathcal{C}} y_c \log \hat{y}_c,$$

在哪里

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{z}) = \frac{\exp(\boldsymbol{z})}{\sum\limits_{c\in\mathcal{C}} \exp(z_c)},$$

和

$$\boldsymbol{z} = f_{\text{exit}_n}(\boldsymbol{x}; \theta),$$

其中 f_{exit_n} 是表示从入口点到第 n 个出口分支的神经网络层的计算的函数，而 θ 表示网络参数，例如这些层的权重和偏差。

为了训练 DDNN，我们形成一个联合优化问题，即最小化每个损失函数的加权和

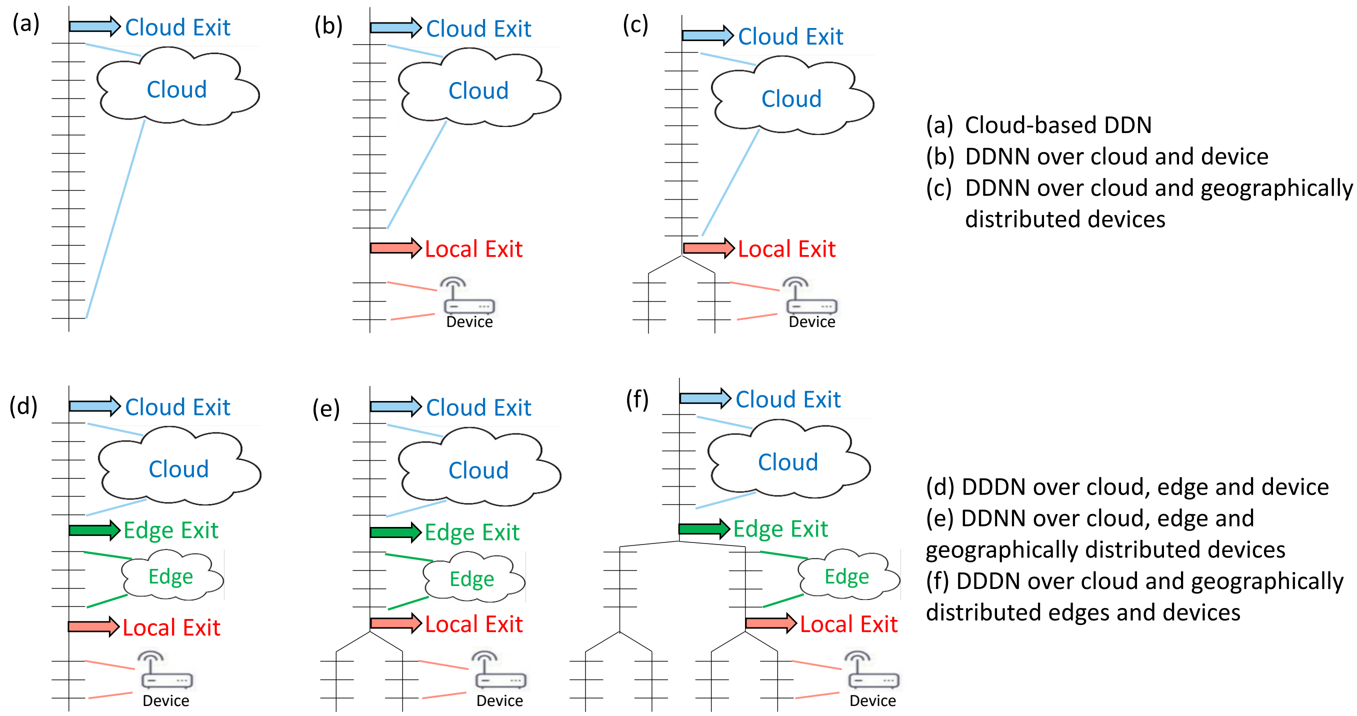

Figure 2. Overview of the DDNN architecture. The vertical lines represent the DNN pipeline, which connects the horizontal bars (NN layers). (a) is the standard DNN (processed entirely in the cloud), (b) introduces end devices and a local exit point that may classify samples before the cloud, (c) extends (b) by adding multiple end devices which are aggregated together for classification, (d) and (e) extend (b) and (c) by adding edge layers between the cloud and end devices, and (f) shows how the edge can also be distributed like the end devices.

exit:

$$L(\hat{\boldsymbol{y}}, \boldsymbol{y}; \theta) = \sum_{n=1}^{N} w_n L(\hat{\boldsymbol{y}}_{\text{exit}_n}, \boldsymbol{y}; \theta),$$

where N is the total number of exit points and w_n is the associated weight of each exit. Equal weights are used for the experimental results of this paper.

D. DDNN Inference

Inference in DDNN is performed in several stages using multiple preconfigured exit thresholds $\boldsymbol{T}$ (one element T at each exit point) as a measure of confidence in the prediction of the sample. One way to define $\boldsymbol{T}$ is by searching over the ranges of $\boldsymbol{T}$ on a validation set and pick the one with the best accuracy. We use a normalized entropy threshold as the confidence criteria (instead of unnormalized entropy as used in [3]) that determines whether to classify (exit) a sample at a particular exit point. The normalized entropy is defined as

$$\eta(\mathbf{x}) = -\sum_{i=1}^{|\mathcal{C}|} \frac{x_i \log x_i}{\log |\mathcal{C}|},$$

where $\mathcal{C}$ is the set of all possible labels and $\mathbf{x}$ is a probability vector. This normalized entropy η has values between 0 and 1 which allows easier interpretation and searching of its corresponding threshold T. For example, η close to 0 means that the DDNN is confident about the prediction of the sample; η close to 1 means it is not confident. At each exit point, η is computed and compared against T in order to determine if the sample should exit at that point.

At a given exit point, if the predictor is not confident in the result (*i.e.*, $\eta > T$), the system falls back to a higher exit point in the hierarchy until the last exit is reached which always performs classification.

We now provide an example of the inference procedure for a DDNN which has multiple end devices and three exit points (configuration (e) in Figure 2):

1) Each end device first sends summary information to local aggregator.
2) The local aggregator determines if the combined summary information is sufficient for accurate classification.
3) If so, the sample is classified (exited).
4) If not, each device sends more detailed information to the edge in order to perform further processing for classification.
5) If the edge believes it can correctly classify the sample it does so and no information is sent to the cloud.
6) Otherwise, the edge forwards intermediate computation to the cloud which makes the final classification.

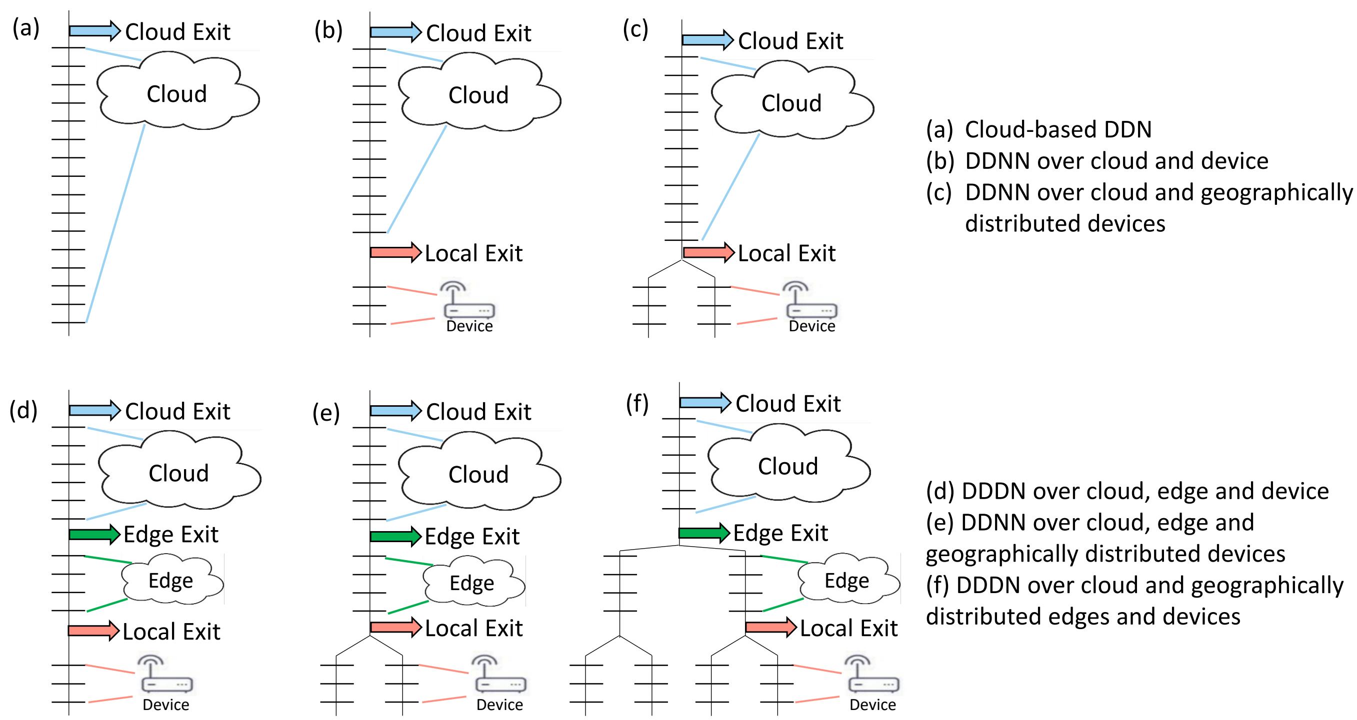

图 2.DDNN 架构概述。垂直线代表 DNN 管道，它连接水平条（NN 层）。(a) 是标准 DNN（完全在云中处理），(b) 引入终端设备和一个可以在云之前对样本进行分类的本地出口点，(c) 通过添加聚合在一起进行分类的多个终端设备来扩展 (b)，(d) 和 (e) 通过在云和终端设备之间添加边缘层来扩展 (b) 和 (c)，(f) 显示边缘如何像终端设备一样分布。

出口：

$$L(\hat{\boldsymbol{y}}, \boldsymbol{y}; \theta) = \sum_{n=1}^{N} w_n L(\hat{\boldsymbol{y}}_{\text{exit}_n}, \boldsymbol{y}; \theta),$$

其中 N 是出口点的总数，w_n 是每个出口的相关权重。本文的实验结果采用等权重。

D. DDNN Inference

DDNN 中的推理是在多个阶段中执行的，使用多个预配置的退出阈值 $\boldsymbol{T}$（每个退出点）处的一个元素 T 作为样本预测置信度的度量。定义 $\boldsymbol{T}$ 的一种方法是在验证集上搜索 $\boldsymbol{T}$ 的范围，并选择准确度最高的一个。我们使用归一化熵阈值作为置信标准（而不是[3]中使用的非归一化熵）来确定是否在特定退出点对样本进行分类（退出）。归一化熵定义为

$$\eta(\mathbf{x}) = -\sum_{i=1}^{|\mathcal{C}|} \frac{x_i \log x_i}{\log |\mathcal{C}|},$$

其中 $\mathcal{C}$ 是所有可能标签的集合，x 是概率向量。该归一化熵 η 的值介于 0 和 1 之间，这使得可以更轻松地解释和搜索

其对应的阈值 T。例如，η接近0意味着DDNN对样本的预测有信心；η接近1表示不自信。在每个退出点，计算 η 并将其与 T 进行比较，以确定样本是否应在该点退出。

在给定的出口点，如果预测器对结果（*i.e.*，$\eta > T$）不信任，系统将回退到层次结构中更高的出口点，直到到达始终执行分类的最后一个出口。

现在，我们提供一个 DDNN 推理过程的示例，该 DDNN 具有多个终端设备和三个出口点（图 2 中的配置 (e)）：

1) 每个终端设备首先将摘要信息发送到本地聚合器。 2）本地聚合器确定组合的摘要信息是否足以进行准确的分类。 3) 如果是，则对样本进行分类（退出）。 4）如果不是，每个设备都会向边缘发送更详细的信息，以便执行进一步的分类处理。 5) 如果边缘认为它可以正确地对样本进行分类，那么它就会这样做，并且不会将任何信息发送到云端。 6）否则，边缘将中间计算转发到云，云进行最终分类。

E. Communication Cost of DDNN Inference

The total communication cost for an end device with the local and cloud aggregator is calculated as

$$c = 4 \times |\mathcal{C}| + (1 - l)\frac{f \times o}{8} \quad (1)$$

where l is the percentage of samples exited locally, $\mathcal{C}$ is the set of all possible labels (3 in our experiments), f is the number of filters, and o is the output size of a single filter for the final NN layer on the end-device. The constant 4 corresponds to 4 bytes which are used to represent a floating-point number and the constant 8 corresponds to bits used to express a byte output. The first term assumes a single floating-point per class, which conveys the probability that the sample to be transmitted from the end device to the local aggregator belongs to this class. This step happens regardless of whether the sample is exited locally or at a later exit point. The second term is the communication between end device and cloud which happens $(1 - l)$ fraction of the time, when the sample is exited in the cloud rather than locally.

F. Accuracy Measures

Throughout the evaluation in Section IV, we use different accuracy measures for the various exit points in a DDNN as follows:

- *Local Accuracy* is the accuracy when exiting 100% of samples at the local exit of a DDNN.
- *Edge Accuracy* is the accuracy when exiting 100% of samples at the edge exit of a DDNN.
- *Cloud Accuracy* is the accuracy when exiting 100% of samples at the cloud exit of a DDNN.
- *Overall Accuracy* is the accuracy when exiting some percentage of samples at each exit point in the hierarchy. The samples classified at each exit point are determined by the entropy threshold T for that exit. The impact of T on classification accuracy and communication cost is discussed in Section IV-D.
- *Individual Accuracy* is the accuracy of an end device NN model trained separately from DDNN. The NN model for each end device consists of a ConvP block followed by a FC block (a single end device portion as shown in Figure 4). In the evaluation, individual accuracy for each device is computed by classifying all samples using the individual NN model and not relying on the local or cloud exit points of a DDNN.

IV. DDNN System Evaluation

In this section, we evaluate DDNN on a scenario with multiple end devices and demonstrate the following characteristics of the approach:

- DDNNs allow multiple end devices to work collaboratively in order to improve accuracy at both the local and cloud exit points.
- DDNNs seamlessly extend the capability of end devices by offloading difficult samples to the cloud.
- DDNNs have built-in fault tolerance. We illustrate that missing any single end device does not dramatically affect the accuracy of the system. Additionally, we show how performance gradually degrades as more end devices are lost.
- DDNNs reduce communication costs for end devices compared to traditional system that offloads all input sensor data to the cloud.

We first introduce the DDNN architecture and dataset used in our evaluation.

A. DDNN Evaluation Architecture

To accommodate the small memory size of the end devices, we use Binary Neural Network [4] blocks[3]. We make use of two types of blocks in [5]: the fused binary fully connected (FC) block and fused binary convolution-pool (ConvP) block as shown in Figure 3. FC blocks each consist of a fully connected layer with m nodes for some m, batch normalization and binary activation. ConvP blocks each consist of a convolutional layer with f filters for some f, a pooling layer and batch normalization and binary activation. A convolution layer has a kernel of size 3x3 with stride 1 and padding 1. A pooling layer has a kernel of size 3x3 with stride 2 and padding 1.

For our experiments, we use version (c) from Figure 2, with six end devices. The system presented can be generalized to a more elaborated structure which includes an edge layer, as shown in (d), (e) or (f) of Figure 2. Figure 4 depicts a detailed view of the DDNN system used in our experiments. In this system, we have six end devices shown in red, a local aggregator, and a cloud aggregator. During training, output from each device is aggregated together at each exit point using one of the aggregation schemes described in Section III-B. We provide detailed analysis on the impact of aggregation schemes at both the local and cloud exit points in Section IV-C. All DDNNs in our experiments are trained with Adam [17] using the following hyper-parameter settings: α of 0.001, β_1 of 0.9, β_2 of 0.999, and ϵ of 1e-8. We train each DDNN for 100 epochs. When training the DDNN, we use equal weights for the local and cloud exit points. We explored heavily weighting both the local exit and the cloud exit, but neither weighting scheme significantly changed the accuracy of the system. This indicates that this solution to the dataset and the problem we are exploring is not sensitive to the weights, but this may not be true for other datasets and problems[4].

[3]A block consists of one or more conventional NN layers

[4]In GoogleNet [9], a less than 1% difference in accuracy was observed based on the values of the weight parameters

E. Communication Cost of DDNN Inference

终端设备与本地和云聚合器的总通信成本计算如下

$$c = 4 \times |\mathcal{C}| + (1 - l)\frac{f \times o}{8} \tag{1}$$

其中 l 是本地退出的样本的百分比，$\mathcal{C}$ 是所有可能标签的集合（在我们的实验中为 3），f 是过滤器的数量，o 是终端设备上最终 NN 层的单个过滤器的输出大小。常量 4 对应于用于表示浮点数的 4 个字节，常量 8 对应于用于表示字节输出的位。第一项假设每个类别有一个浮点，它表示从终端设备传输到本地聚合器的样本属于该类别的概率。无论样本是在本地退出还是在稍后的退出点退出，都会发生此步骤。第二项是终端设备和云之间的通信，当样本在云中而不是本地存在时，这种通信在 $(1 - l)$ 的一小部分时间内发生。

F. Accuracy Measures

在第四节的整个评估过程中，我们对 DDNN 中的各个出口点使用不同的精度度量，如下所示：

- *Local Accuracy* 是在 DDNN 本地出口处退出 100% 样本时的准确率。
- *Edge Accuracy* 是在 DDNN 边缘出口处退出 100% 样本时的准确率。
- *Cloud Accuracy* 是 DDNN 云出口处 100% 样本退出时的准确率。
- *Overall Accuracy* 是在层次结构中每个退出点退出一定百分比的样本时的准确性。在每个出口点分类的样本由该出口的熵阈值 T 确定。第 IV-D 节讨论了 T 对分类准确性和通信成本的影响。
- *Individual Accuracy* 是与 DDNN 分开训练的终端设备 NN 模型的准确性。每个终端设备的 NN 模型由 ConvP 块和 FC 块组成（单个终端设备部分，如图 4 所示）。在评估中，每个设备的单独精度是通过使用单独的 NN 模型对所有样本进行分类来计算的，而不依赖于 DDNN 的本地或云出口点。

四. DDNN系统评估

在本节中，我们在具有多个终端设备的场景中评估 DDNN，并演示该方法的以下特征：

- DDNN 允许多个终端设备协作工作，以提高本地和云出口点的准确性。
- DDNN 通过将困难的样本卸载到云端来无缝扩展终端设备的功能。
- DDNN 具有内置的容错能力。我们证明，丢失任何单个终端设备不会显着影响系统的准确性。此外，我们还展示了随着更多终端设备丢失，性能如何逐渐下降。
- 与将所有输入传感器数据卸载到云端的传统系统相比，DDNN 降低了终端设备的通信成本。

我们首先介绍评估中使用的 DDNN 架构和数据集。

A. DDNN Evaluation Architecture

为了适应终端设备的小内存，我们使用二元神经网络[4]块[3]。我们在[5]中使用两种类型的块：融合二进制全连接（FC）块和融合二进制卷积池（ConvP）块，如图3所示。每个FC块都由一个全连接层组成，其中带有一些m的m节点，批量归一化和二进制激活。每个 ConvP 块都包含一个带有针对某些 f 的 f 过滤器的卷积层、一个池化层以及批量归一化和二进制激活。卷积层的内核大小为 3x3，步长为 1，填充为 1。池化层的内核大小为 3x3，步长为 2，填充为 1。

在我们的实验中，我们使用图 2 中的版本 (c)，有六个终端设备。所提出的系统可以推广到更复杂的结构，其中包括边缘层，如图 2 的 (d)、(e) 或 (f) 所示。图 4 描绘了我们实验中使用的 DDNN 系统的详细视图。在此系统中，我们有六个以红色显示的终端设备、一个本地聚合器和一个云聚合器。在训练期间，每个设备的输出在每个出口点使用第 III-B 节中描述的聚合方案之一聚合在一起。我们在第 IV-C 节中详细分析了本地和云出口点聚合方案的影响。我们实验中的所有 DDNN 均使用 Adam [17] 使用以下超参数设置进行训练：α 为 0.001，β_1 为 0.9，β_2 为 0.999，ϵ 为 1e-8。我们将每个 DDNN 训练 100 个 epoch。在训练 DDNN 时，我们对本地出口点和云端出口点使用相同的权重。我们探索了对本地出口和云出口进行大量加权，但这两种加权方案都没有显着改变系统的准确性。这表明我们正在探索的数据集和问题的这种解决方案对权重不敏感，但对于其他数据集和问题可能并非如此[4]。

[3]A block consists of one or more conventional NN layers

[4]In GoogleNet [9], a less than 1% difference in accuracy was observed based on the values of the weight parameters

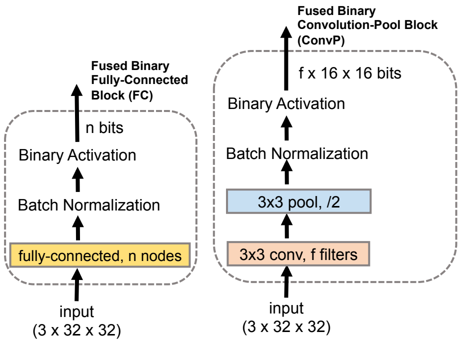

Figure 3. Fused binary blocks consisting of one or more standard NN layers. The fused binary fully connected (FC) block is a fully connected layer with n nodes, batch normalization and binary activation. The fused binary convolution-pool (ConvP) block consists of a convolutional layer with f filters, a pooling layer, batch normalization and binary activation. The convolution layer has a kernel of size 3x3 with stride 1 and padding 1. The pooling layer has a kernel of size 3x3 with stride 2 and padding 1. These blocks are used as they are presented in [5].

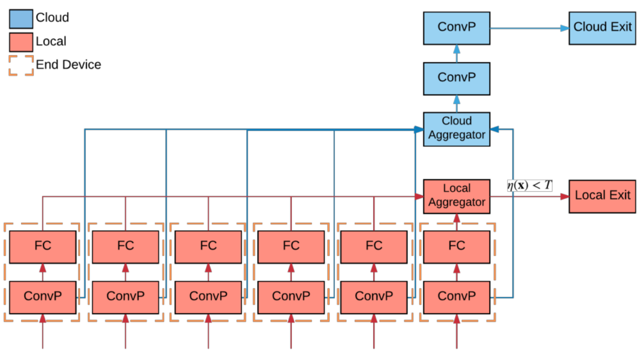

Figure 4. The DDNN architecture used in the system evaluation. The FC and ConvP blocks in red and blue correspond to layers run on end devices and the cloud respectively. The dashed orange boxes represent the end devices and show which blocks of the DDNN are mapped onto each device. The local aggregator shown in red combines the exit output (a short vector with length equal to the number of classes) from each end device in order to determine if local classification for the given input sample can be performed accurately. If the local exit is not confident (*i.e.* $\eta(x) > T$), the activation output after the last convolutional layer from each end device is sent to the cloud aggregator (shown in blue), which aggregates the input from each device, performs further NN layer processing, and outputs a final classification result. The aggregation of input for multiple end devices is discussed in Section IV-C.

B. Multi-view Multi-camera Dataset

We evaluate the proposed DDNN framework on a multi-view multi-camera dataset [18]. This dataset consists of images acquired at the same time from six cameras placed at different locations facing the same general area. For the purpose of our evaluation, we assume that each camera is attached to an end device, which can transmit the captured images over a bandwidth-constraint wireless network to a physical endpoint connected to the cloud.

The dataset provides object bounding box annotations. Multiple bounding boxes may exist in a single image, each of which corresponds to a different object in the frame. In

Figure 5. Example images of three objects (person, bus, car) from the multi-view multi-camera dataset. The six devices (each with their own camera) capture the same object from different orientations. An all grey image denotes that the object is not present in the frame.

preparing the dataset, for each bounding box, we extract an image, and manually synchronize[5] the same object across the multiple devices that the object appears in for the given frame. Examples of the extracted images are shown in Figure 5. Each row corresponds to a single sample used for classification. We resize each extracted sample to a 32x32 RGB pixel image. For each device that a given object does not appear in, we use a blank image and assign a label of -1, meaning that the object is not present in the frame. Labels 0, 1, and 2 correspond to car, bus and person, respectively. Objects that are not present in a frame (*i.e.*, label of -1) are not used during training. We split the dataset into 680 training samples and 171 testing samples. Figure 6 shows the distribution of samples at each device. Due to the imbalanced number of class samples in the dataset, the individual accuracy of each end device differs widely, as shown by the "Individual" curve of Figure 8. A full description of the training process for the individual NN models is provided in Section IV-E. The processed dataset used in this paper is available at [20].

C. Impact of Aggregation Schemes

In order to perform classification on the input from multiple end devices, we must aggregate the information from each end device. We consider three aggregation methods (max pooling, average pooling, and concatenation) outlined in Section III-B, at both the local and cloud exit points. The accuracy of different aggregation schemes are shown in Table I. The first two letters identify the local aggregation scheme and the last two letters identify the scheme used by the cloud aggregator. For example, MP-CC means the local aggregator uses max-pooling and the cloud uses concatenation. Recall that each input to the local aggregator is a floating-point vector of length equal to the number of classes (corresponding to the output from the final FC block for a single device as shown in Figure 4) and the device

[5]In practical object tracking systems, this synchronization step is typically automated [19].

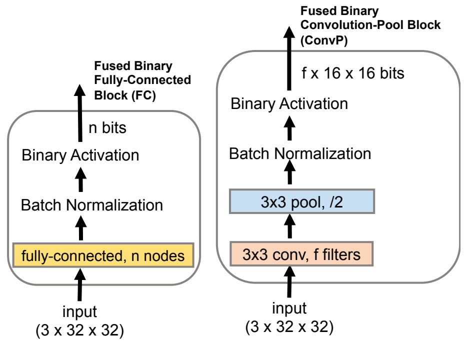

图 3. 由一个或多个标准 NN 层组成的融合二进制块。融合的二进制全连接（FC）块是一个具有 n 节点、批量归一化和二进制激活的全连接层。融合二进制卷积池 (ConvP) 块由带有 f 滤波器的卷积层、池化层、批量归一化和二进制激活组成。卷积层有一个大小为 3x3、步幅为 1、填充为 1 的内核。池化层有一个大小为 3x3、步幅为 2、填充为 1 的内核。这些块的使用方式如 [5] 中所示。

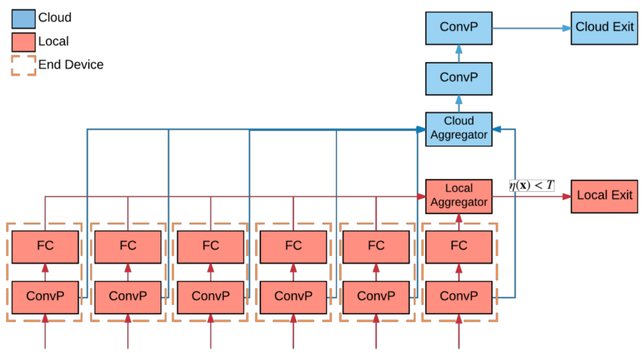

图 4.系统评估中使用的 DDNN 架构。红色和蓝色的 FC 和 ConvP 块分别对应于在终端设备和云上运行的层。橙色虚线框代表终端设备，并显示 DDNN 的哪些块映射到每个设备。以红色显示的本地聚合器组合来自每个终端设备的退出输出（长度等于类数的短向量），以确定是否可以准确地执行给定输入样本的本地分类。如果本地出口不可信（*i.e.* $\eta(x) > T$），则每个终端设备的最后一个卷积层之后的激活输出将被发送到云聚合器（以蓝色显示），云聚合器聚合来自每个设备的输入，执行进一步的 NN 层处理，并输出最终的分类结果。多个终端设备的输入聚合将在第 IV-C 节中讨论。

B. *Multi-view Multi-camera Dataset*

我们在多视图多摄像机数据集上评估了所提出的 DDNN 框架 [18]。该数据集由放置在面向同一区域的不同位置的六个摄像机同时获取的图像组成。出于评估目的，我们假设每个摄像头都连接到一个终端设备，该终端设备可以通过带宽受限的无线网络将捕获的图像传输到连接到云的物理端点。

数据集提供对象边界框注释。单个图像中可能存在多个边界框，每个边界框对应于帧中的不同对象。在

图 5. 多视图多摄像头数据集中的三个对象（人、公共汽车、汽车）的示例图像。这六个设备（每个设备都有自己的摄像头）从不同方向捕获同一对象。全灰色图像表示该对象不存在于帧中。

准备数据集，对于每个边界框，我们提取图像，并在给定帧中出现该对象的多个设备上手动同步[5]同一对象。提取图像的示例如图 5 所示。每一行对应于用于分类的单个样本。我们将每个提取的样本的大小调整为 32x32 RGB 像素图像。对于给定对象未出现的每个设备，我们使用空白图像并分配标签 -1，这意味着该对象不存在于帧中。标签 0、1 和 2 分别对应汽车、公共汽车和人。训练期间不会使用帧中不存在的对象（*i.e.*，标签为 - 1）。我们将数据集分为 680 个训练样本和 171 个测试样本。图 6 显示了每个设备上的样本分布。由于数据集中类样本数量不平衡，每个终端设备的个体精度差异很大，如图 8 的“个体”曲线所示。第 IV-E 节提供了各个 NN 模型训练过程的完整描述。本文使用的处理后的数据集可在[20]中获得。

C. *Impact of Aggregation Schemes*

为了对来自多个终端设备的输入进行分类，我们必须聚合来自每个终端设备的信息。我们在本地和云出口点考虑第 III-B 节中概述的三种聚合方法（最大池化、平均池化和串联）。不同聚合方案的准确性如表一所示。前两个字母表示本地聚合方案，最后两个字母表示云聚合器使用的方案。例如，MP-CC 意味着本地聚合器使用最大池化，而云使用串联。回想一下，本地聚合器的每个输入都是一个浮点向量，其长度等于类的数量（对应于单个设备的最终 FC 块的输出，如图 4 所示）和设备

[5]In practical object tracking systems, this synchronization step is typically automated [19].

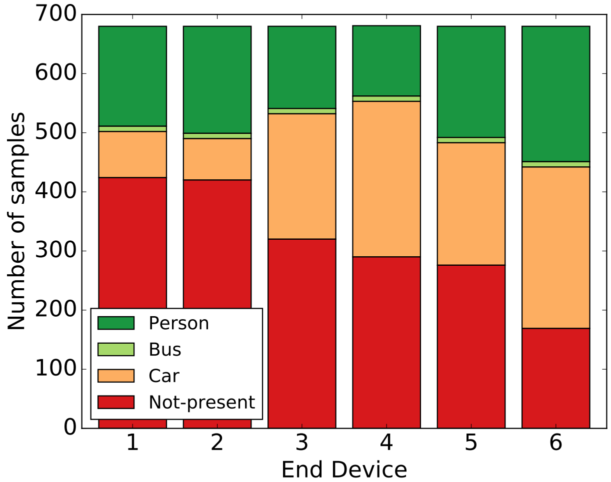

Figure 6. The distribution of class samples for each end device in the multi-view multi-camera dataset.

Table I
ACCURACY OF AGGREGATION SCHEMES. THE FIRST TWO LETTERS IDENTIFY THE LOCAL AGGREGATION SCHEME, AND THE LAST TWO LETTERS IDENTIFY THE CLOUD AGGREGATION SCHEME. FOR EXAMPLE, MP-CC MEANS THE LOCAL AGGREGATOR USES MAX-POOLING AND THE CLOUD AGGREGATOR USES CONCATENATION. THE ACCURACY OF EACH EXIT POINT (EITHER LOCAL OR CLOUD) IS COMPUTED USING THE ENTIRE TEST SET. IN PRACTICE, WE WILL EXIT A PORTION OF SAMPLES LOCALLY BASED ON THE ENTROPY THRESHOLD T AND SEND THE REMAINING SAMPLES IN THE CLOUD. DUE TO ITS HIGH PERFORMANCE, MP-CC IS USED IN THE REMAINING EXPERIMENTS OF THIS PAPER.

Schemes	**Local Acc. (%)**	**Cloud Acc. (%)**
MP-MP	95	91
MP-CC	**98**	**98**
AP-AP	86	98
AP-CC	75	96
CC-CC	85	94
AP-MP	88	93
MP-AP	89	97
CC-MP	77	87
CC-AP	80	94

output sent to the cloud aggregator is the output from the final ConvP block.

The MP-MP scheme has good classification accuracy for the local aggregator but poor performance in the cloud. The elements in the vectors at the local aggregator correspond to the same features (*e.g.*, the first item is the likelihood that the input corresponds to that class). Therefore, max pooling corresponds to taking the max response for each class over all end devices, and shows good performance. On the other hand, since the information sent from the end devices to the cloud is the activation output from the filters at each device, which corresponds to different visual features in the input from the viewpoint of each individual end device, max pooling these features does not perform well.

Comparing MP-MP and MP-CC schemes, though both use MP for local aggregators, MP-CC increases the accuracy of the local classifier. In the training phrase, during backpropagation the MP-MP scheme only passes gradients through a device that gives the highest response while MP-CC scheme passes gradients through all devices. Therefore, using CC aggregator in the cloud allows all devices to learn better filters (filter weights) that give a stronger response for the local MP aggregator, resulting in a better classification accuracy.

The CC-CC scheme shows an opposite trend where the local accuracy is poor while the cloud accuracy is high. Concatenating the local information (instead of a pooling scheme), does not enforce any relationship between output for the same class on multiple devices and therefore performs worse. Concatenating the output for the cloud aggregator maintains the most information for NN layer processing in the cloud and therefore performs well.

Generally, for the local aggregator, average pooling performs worse than max pooling. This is because some of the end devices do not have the object present in the given frame. Average pooling take average of all outputs from end devices; this compromises the strong outputs from end devices in which the object is present. Based on these results, we use the MP-CC aggregation scheme throughout the paper.

D. Entropy Threshold

The entropy threshold for an exit point, T, corresponds to the level of confidence that is required in order to exit a sample. A threshold value of 0 would mean that no samples will exit and a value of 1 would mean that all samples exit at that point. Figure 7 shows the relationship between T at the local aggregator and the overall accuracy of the DDNN. We observe that as more samples are exited at the local exit, the overall accuracy decreases. This is expected, as the accuracy of the local exit is typically lower than that of the cloud exit.

We need to set the threshold appropriately to achieve a balance between the communication cost, as defined in Section III-E, latency and accuracy of the system. In this case, we see that setting the threshold to 0.8 results in the best overall accuracy with significantly reduced communication, *i.e.*, 97% accuracy while exiting 60.82% of samples locally as shown in Table II where in addition to local exit (%) and overall accuracy (%), communication cost in bytes is given. We set $T = 0.8$ for the remaining experiments in the system evaluation, unless noted otherwise.

The local classifier may do better than cloud for certain samples where low-level features are more robust in classification than higher-level features. By setting an appropriate threshold T, we can improve overall accuracy. In this experiment, $T = 0.8$ corresponds to that sweet spot where some samples which are incorrectly classified by the cloud classifier can actually be correctly classified by the local classifier. Such a threshold indicates the optimal point where both local and cloud classifier work best together.

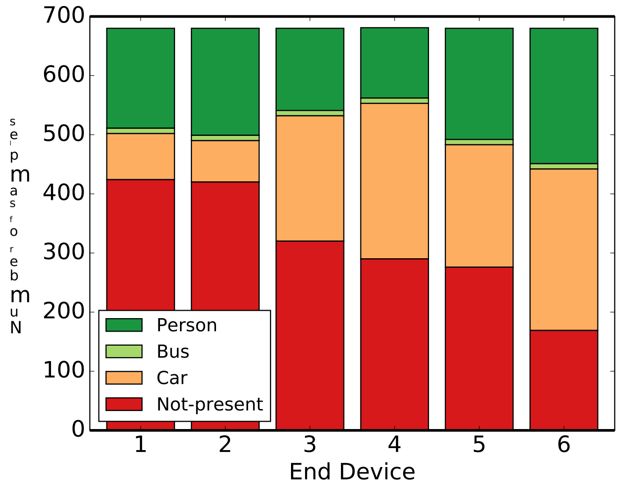

图 6. 多视图多摄像头数据集中每个终端设备的类样本分布。

表一
聚合方案的准确性。前两个字母标识本地聚合方案，最后两个字母标识云聚合方案。例如，MP-CC 表示本地聚合器使用最大池化，而云聚合器使用串联。每个退出点（本地或云）的准确性是使用整个测试集计算的。在实践中，我们会根据熵阈值T在本地退出部分样本，并将剩余样本发送到云端。由于其高性能，本文的其余实验中均使用 MP-CC。

Schemes	Local Acc. (%)	Cloud Acc. (%)
MP-MP	95	91
MP-CC	**98**	**98**
AP-AP	86	98
AP-CC	75	96
CC-CC	85	94
AP-MP	88	93
MP-AP	89	97
CC-MP	77	87
CC-AP	80	94

发送到云聚合器的输出是最终 ConvP 块的输出。

MP-MP 方案对于本地聚合器具有良好的分类精度，但在云中性能较差。本地聚合器向量中的元素对应于相同的特征（*e.g.*，第一项是输入对应于该类的可能性）。因此，最大池化对应于在所有终端设备上获取每个类的最大响应，并显示出良好的性能。另一方面，由于从终端设备发送到云端的信息是每个设备上的过滤器的激活输出，从每个终端设备的角度来看，它对应于输入中的不同视觉特征，因此最大池化这些特征的效果并不好。

比较 MP-MP 和 MP-CC 方案，虽然两者都使用 MP 作为本地聚合器，但 MP-CC 提高了精度本地分类器的有效性。在训练阶段，在反向传播过程中，MP-MP 方案仅通过给出最高响应的设备传递梯度，而 MP-CC 方案则通过所有设备传递梯度。因此，在云中使用 CC 聚合器可以让所有设备学习更好的过滤器（过滤器权重），从而为本地 MP 聚合器提供更强的响应，从而获得更好的分类精度。

CC-CC方案则呈现出相反的趋势，本地精度较差，而云端精度较高。连接本地信息（而不是池方案）不会强制多个设备上同一类的输出之间存在任何关系，因此性能较差。连接云聚合器的输出可以在云中保留最多的 NN 层处理信息，因此性能良好。

一般来说，对于本地聚合器，平均池化的性能比最大池化差。这是因为某些终端设备在给定帧中不存在该对象。平均池取终端设备所有输出的平均值；这会损害存在该物体的终端设备的强大输出。基于这些结果，我们在整篇论文中使用 MP-CC 聚合方案。

D. Entropy Threshold

退出点的熵阈值 T 对应于退出样本所需的置信水平。阈值 0 表示没有样本将退出，值为 1 表示所有样本在该点退出。图 7 显示了本地聚合器处的 T 与 DDNN 整体精度之间的关系。我们观察到，随着更多的样本在本地出口处退出，整体精度会下降。这是预期的，因为本地出口的准确度通常低于云出口的准确度。

我们需要适当设置阈值，以实现通信成本（如第 III-E 节中定义）、系统延迟和准确性之间的平衡。在这种情况下，我们看到，将阈值设置为 0.8 会产生最佳的总体精度，同时显着减少通信，*i.e.*，精度为 97%，同时本地退出 60.82% 的样本，如表 II 所示，其中除了本地退出 (%) 和总体精度 (%) 之外，还给出了以字节为单位的通信成本。除非另有说明，我们为系统评估中的其余实验设置 $T = 0.8$。

对于某些样本，本地分类器可能比云分类器做得更好，其中低级特征比高级特征在分类上更稳健。通过设置适当的阈值 T，我们可以提高整体准确性。在此实验中，$T = 0.8$ 对应于最佳点，其中一些被云分类器错误分类的样本实际上可以被本地分类器正确分类。这样的阈值表示本地分类器和云分类器协同工作的最佳点。

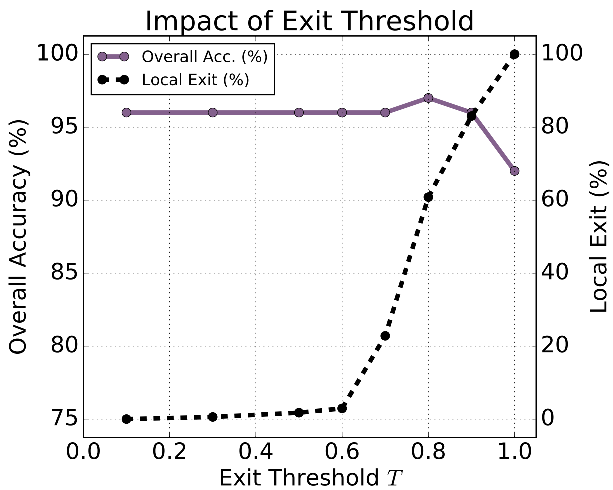

Figure 7. Overall accuracy of the system as the entropy threshold for the local exit is varied from 0 to 1. For this experiment, 4 filters are used in the ConvP blocks on the end devices.

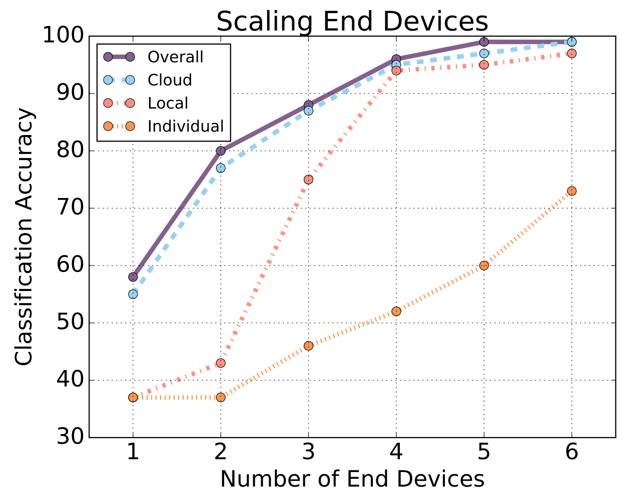

Figure 8. Accuracy of the DDNN system as additional end devices are added. The accuracy of "Overall" is obtained by exiting a percentage of the samples locally and the rest in the cloud. The accuracy of "Cloud" and "Local" are computed by exiting all samples at each point, respectively. The end devices are ordered by their "Individual" classification accuracy, sorted from worst to best.

Table II
EFFECTS OF DIFFERENT EXIT THRESHOLD (T) SETTINGS FOR THE LOCAL EXIT. $T = 0.8$ IS USED IN THE REMAINING EXPERIMENTS.

T	Local Exit (%)	Overall Acc. (%)	Comm. (B)
0.1	0.00	96	140
0.3	0.58	96	139
0.5	1.75	96	138
0.6	2.92	96	136
0.7	22.81	96	111
0.8	**60.82**	**97**	**62**
0.9	83.04	96	34
1.0	100.00	92	12

E. Impact of Scaling Across End Devices

In order to scale DDNNs across multiple end devices, we distribute the lower sections of Figure 4, shown in red, over the corresponding devices, outlined in orange. Figure 8 shows how the accuracy of the system improves as additional end devices (each with its attached input cameras) are added. The devices are added in order sorted by their individual accuracy from worst to best (*i.e.*, the device with the lowest accuracy first and the device with the highest accuracy last).

The first observation is the large variation in the individual accuracy of the end devices, as noted earlier. Due to the nature of the dataset, some devices are naturally better positioned and generally have clearer observations of the objects. Looking at the viewpoints of each camera in Figure 5, we see that the selected examples for Device 6 have clear frontal views of each object. This viewpoint gives Device 6 the highest individual accuracy at over 70%. By comparison, Device 2 has the lowest individual accuracy at under 40%.

The "Local" and "Cloud" curves show the accuracy of the system at each exit point when all samples are exited at that point. We observe that the cloud exit point outperforms the local exit point at all numbers of end devices. The gap is widest when there are fewer devices. This suggests that the additional NN layers in the cloud significantly improve the final classification result when the problem is more difficult due to limited labeled training data for an end device. Once all six end devices are added, both the local and cloud aggregators have high accuracy. The "Overall" curve represents the overall accuracy of the system when the threshold for the local exit point is set to 0.8. We see that this curve is roughly equivalent to exiting all samples at the cloud (but at a much reduced communication cost as 60.82% of samples are exited locally). Generally, these results show that by combining multiple viewpoints we can increase the classification accuracy at both the local and cloud level by a substantial margin when compared to the individual accuracy of any device. The resulting accuracy of the DDNN system is superior to any individual device accuracy by over 20%. Moreover, we note that the 60.82% of samples which exit locally enjoy lowered latency in response time.

F. Impact of Cloud Offloading on Accuracy Improvements

DDNNs improve the overall accuracy of the system by offloading difficult samples to the cloud, which perform further NN layer processing and final classification. Figure 9 shows the accuracy and communication costs of DDNN as the number of filters on the end devices increases. For all settings, the NN layers stored on an end device require under 2 KB of memory. In this experiment, we configure the local exit threshold T such that around 75% of samples are exited locally and around 25% of samples are offloaded

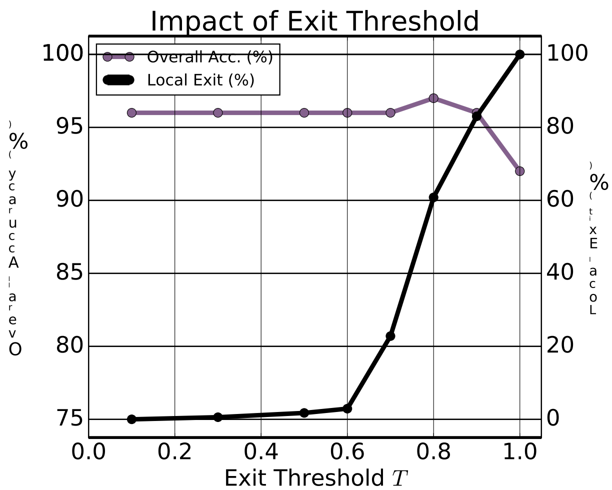

图 7. 当本地出口的熵阈值从 0 到 1 变化时，系统的总体精度。对于此实验，终端设备上的 ConvP 块中使用了 4 个滤波器。

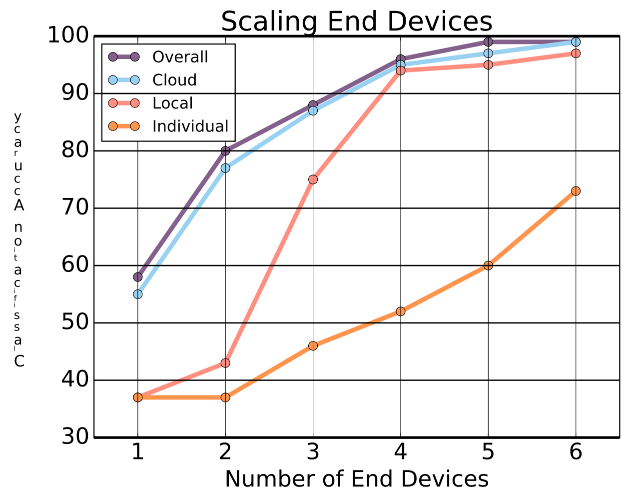

图 8. 添加额外终端设备后 DDNN 系统的准确性。“总体”的准确性是通过在本地退出一定比例的样本并将其余样本留在云端来获得的。“云”和“本地”的准确性是通过分别退出每个点的所有样本来计算的。终端设备按其“单独”分类精度排序，从最差到最好排序。

表 II 不同退出阈值 (T) 设置对本地退出的影响。其余实验中使用 $T = 0.8$。

T	Local Exit (%)	Overall Acc. (%)	Comm. (B)
0.1	0.00	96	140
0.3	0.58	96	139
0.5	1.75	96	138
0.6	2.92	96	136
0.7	22.81	96	111
0.8	**60.82**	**97**	**62**
0.9	83.04	96	34
1.0	100.00	92	12

E. *Impact of Scaling Across End Devices*

为了跨多个终端设备扩展 DDNN，我们将图 4 的下半部分（以红色显示）分布在相应的设备上（以橙色轮廓显示）。图 8 显示了随着附加终端设备（每个设备都附带输入摄像头）的添加，系统的准确性如何提高。设备按照各自的准确度从最差到最好的顺序添加（*i.e.*，首先是准确度最低的设备，最后是准确度最高的设备）。

第一个观察结果是终端设备的个体精度存在巨大差异，如前所述。由于数据集的性质，一些设备自然会处于更好的位置，并且通常可以更清晰地观察对象。查看图 5 中每个摄像机的视点，我们发现设备 6 的所选示例具有每个对象的清晰正面视图。从这个角度来看，Device 6 的个体准确率最高，超过 70%。相比之下，设备 2 的个体准确率最低，低于 40%。

“本地”和“云”曲线显示了当所有样本都在该点退出时，系统在每个退出点的精度。我们观察到云退出点的表现优于所有数量的终端设备上的本地出口点。当设备较少时，差距最大。这表明，当由于终端设备的标记训练数据有限而导致问题变得更加困难时，云中的附加神经网络层可以显着改善最终的分类结果。添加所有六个终端设备后，本地聚合器和云聚合器都具有很高的准确性。“总体”曲线表示当局部出口点的阈值设置为 0.8 时系统的总体精度。我们看到这条曲线大致相当于在云端退出所有样本（但通信成本大大降低，因为 60.82% 的样本在本地退出）。一般来说，这些结果表明，与任何设备的单独精度相比，通过结合多个观点，我们可以大幅提高本地和云级别的分类精度。 DDNN 系统的最终精度比任何单个设备的精度高出 20% 以上。此外，我们注意到，60.82% 的本地退出样本的响应时间延迟较低。

F. *Impact of Cloud Offloading on Accuracy Improvements*

DDNN 通过将困难样本卸载到云端来提高系统的整体准确性，云端执行进一步的神经网络层处理和最终分类。图 9 显示了随着终端设备上的过滤器数量增加，DDNN 的准确性和通信成本。对于所有设置，存储在终端设备上的 NN 层需要不到 2 KB 的内存。在这个实验中，我们配置了本地退出阈值 T，使得大约 75% 的样本在本地退出，大约 25% 的样本被卸载

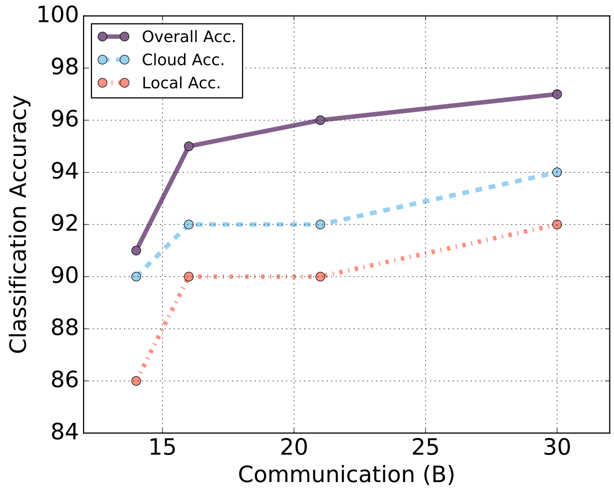

Figure 9. Accuracy and communication cost (in bytes) for increasingly larger end device memory sizes that accommodate additional filters. We notice that cloud offloading leads to improved accuracy.

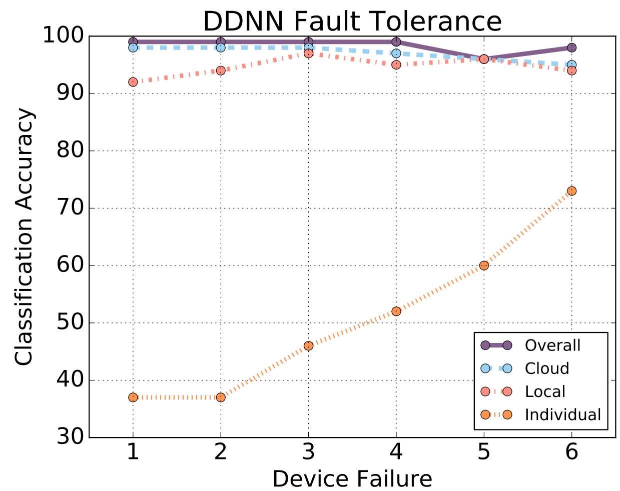

Figure 10. The impact on DDNN system accuracy when any single end device has failed.

to the cloud. We see that DDNNs achieve around a 5% improvement in accuracy compared to using just the local aggregator. This demonstrates the advantage for offloading to the cloud even when larger models (more filters) with improved local accuracy are used on the end devices.

G. Fault Tolerance of DDNNs

A key motivation for distributed systems is fault tolerance. Fault tolerance implies that the system still works well even when some parts are broken. In order to test the fault tolerance of DDNN, we simulate end device failures and look at the resulting accuracy of the system. Figure 10 shows the accuracy of the system under the presence of individual device failures. Regardless of the device that is missing, the system still achieves over a 95% overall classification accuracy. Specifically, even when the device with the highest individual accuracy has failed, which is Device 6, the overall accuracy is reduced by only 3%. This suggests that for this dataset, the automatic fault tolerance provided by DDNN makes the system reliable even in the presence of device failure.

We can also view figure 8 from the perspective of providing fault tolerance for the system. As we decrease the number of end devices from 6 to 4, we observe that the overall accuracy of the system drops only 4%. This suggests that the system can also be robust to mutliple failing end devices.

H. Reducing Communication Costs

DDNNs significantly reduces the communication cost of inference compared to the standard method of offloading raw sensor input to the cloud. Sending a 32x32 RGB pixel image (the input size of our dataset) to the cloud costs 3072 bytes per image sample. By comparison, as shown in Table II, the largest DDNN model used in our evaluation section requires only 140 bytes of communication per sample on average (an over 20x reduction in communication costs). This communication reduction for an end device results from transmitting class-label related intermediate results to the local aggregator for all samples and binarized communication with the cloud when additional NN layer processing is required for classification with improved accuracy.

V. DDNN Provision for Horizontal and Vertical Scaling

The evaluation in the previous section shows that DDNN is able to achieve high overall accuracy through provisioning the network to scale both horizontally, across end devices, and vertically, over the network hierarchy. Specifically, we show that DDNN scales vertically, by exiting easier input samples locally for low-latency response and offloading difficult samples to the cloud for high overall recognition accuracy, while maintaining a small memory footprint on the end devices and incurring a low communication cost. This result is not obvious, as we need sufficiently good feature representations from the lower parts of the DNN (running on the end devices with limited resources) in order for the upper parts of the neural network (running in the cloud) to achieve high accuracy under the low communication cost constraint. Therefore, we show in a positive way that the proposed method of jointly training a single DNN with multiple exit points at each part of the distributed hierarchy allows us to meet this goal. That is, DDNN optimizes the lower parts of the DNN to create a sufficiently good feature representations to support both samples exited locally and those processed further in the cloud.

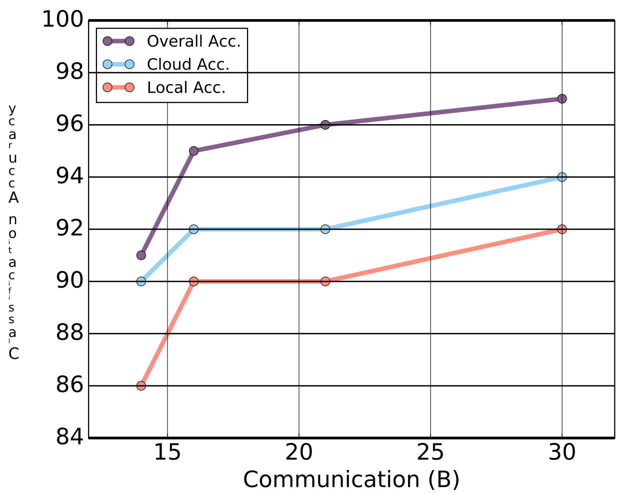

图 9. 容纳额外过滤器的终端设备内存大小越来越大的精度和通信成本（以字节为单位）。我们注意到云卸载可以提高准确性。

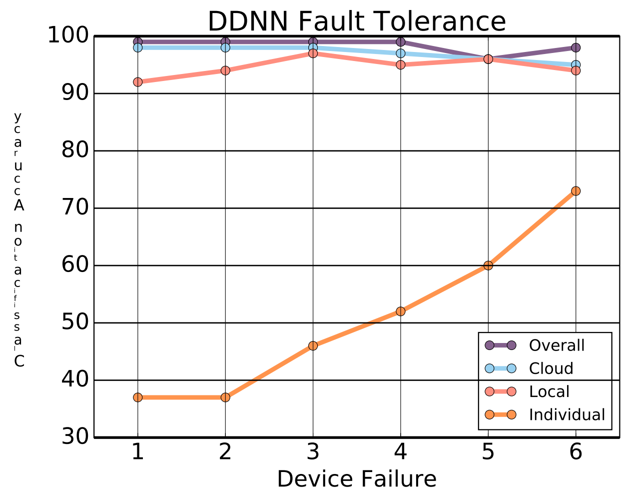

图 10. 任何单端设备发生故障时对 DDNN 系统精度的影响。

到云端。我们发现，与仅使用本地聚合器相比，DDNN 的准确率提高了约 5%。这证明了即使在终端设备上使用具有改进的本地精度的更大模型（更多过滤器）时，卸载到云的优势也是如此。

G. *Fault Tolerance of DDNNs*

分布式系统的一个关键动机是容错。容错意味着即使某些部件损坏，系统仍然可以正常运行。为了测试 DDNN 的容错能力，我们模拟终端设备故障并查看系统的准确性。图 10 显示了在存在单个设备故障的情况下系统的准确性。无论缺少哪个设备，系统仍然可以实现超过 95% 的总体分类精度。具体来说，即使单个精度最高的器件（即器件 6）出现故障，整体精度也仅降低 3%。这表明，对于该数据集，DDNN 提供的自动容错功能使系统即使在存在设备故障的情况下也是可靠的。

我们还可以从为系统提供容错的角度来看待图8。当我们将终端设备的数量从 6 个减少到 4 个时，我们观察到系统的整体精度仅下降了 4%。这表明该系统对于多个出现故障的终端设备也具有鲁棒性。

H. *Reducing Communication Costs*

与将原始传感器输入卸载到云端的标准方法相比，DDNN 显着降低了推理的通信成本。将 32x32 RGB 像素图像（数据集的输入大小）发送到云端需要 3072 字节每个图像样本。相比之下，如表 II 所示，我们的评估部分中使用的最大 DDNN 模型平均每个样本仅需要 140 字节的通信（通信成本降低了 20 倍以上）。终端设备的这种通信减少是由于将所有样本的类标签相关中间结果传输到本地聚合器，并在需要额外的神经网络层处理以提高分类精度时与云进行二进制化通信。

V. DDNN 水平和垂直扩展的规定

上一节中的评估表明，DDNN 能够通过配置网络在跨终端设备的水平扩展和在网络层次结构上的垂直扩展来实现较高的整体精度。具体来说，我们证明了 DDNN 可以垂直扩展，通过在本地退出更容易的输入样本来实现低延迟响应，并将困难的样本卸载到云端以实现较高的整体识别精度，同时在终端设备上保持较小的内存占用并产生较低的通信成本。这个结果并不明显，因为我们需要 DNN 的下部部分（在资源有限的终端设备上运行）提供足够好的特征表示，以便神经网络的上部部分（在云端运行）在低通信成本约束下实现高精度。因此，我们以积极的方式表明，所提出的在分布式层次结构的每个部分联合训练具有多个出口点的单个 DNN 的方法使我们能够实现这一目标。也就是说，DDNN 优化了 DNN 的下部部分，以创建足够好的特征表示，以支持本地存在的样本和在云端进一步处理的样本。

To meet the goal of horizontal scaling, we provide a principled way of jointly training a DNN with inputs from multiple devices through feature pooling via local and cloud aggregators and demonstrate that by aggregating features from each device we can dramatically improve the accuracy of the system both at the local and cloud level. Filters on each device are automatically tuned to process the geographically unique inputs and work together toward to the same overall objective leading to high overall accuracy. Additionally, we show that DDNN provides built-in fault tolerance across the end devices and is still able to achieve high accuracy in the presence of failed devices.

VI. Conclusion

In this paper, we propose a novel distributed deep neural network architecture (DDNN) that is distributed across computing hierarchies, consisting of the cloud, the edge and end devices. We demonstrate for a multi-view, multi-camera dataset that DDNN scales vertically from a few NN layers on end devices or the edge to many NN layers in the cloud and scales horizontally across multiple end devices. The aggregation of information communicated from different devices is built into the joint training of DDNN and is handled automatically at inference time. This approach simplifies the implementation and deployment of distributed cloud offloading and automates sensor fusion and system fault tolerance.

The experimental results suggest that with our DDNN framework, a single DNN properly trained can be mapped onto a distributed computing hierarchy to meet the accuracy, communication and latency requirements of a target application while gaining inherent benefits associated with distributed computing such as fault tolerance and privacy.

DDNNs reduce the required communication compared to a standard cloud offloading approach by exiting many samples at the local aggregator and sending a compact binary feature representation to the cloud when additional processing is required. For our evaluation dataset, the communication cost of DDNN is reduced by a factor of over 20x compared to offloading raw sensor input to a DNN in the cloud which performs all of the inference computation.

DDNN provides a framework for further research in mapping DNN into a distributed computing hierarchy. For future work, we will investigate the performance of DDNNs on applications with a larger dataset with multiple types of input modalities [21] and more end devices. Currently, all layers in DDNN are binary. While binary layers are a requirement for end devices due to the limited space on devices, it is not necessary in the cloud. We will explore other types of aggregation schemes and mixed precisions schemes where the end devices use binary NN layers and the cloud uses mixed-precision or floating-point NN layers.

Acknowledgment

This work is supported in part by gifts from the Intel Corporation and in part by the Naval Supply Systems Command award under the Naval Postgraduate School Agreements No. N00244-15-0050 and No. N00244-16-1-0018.

References

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] K. Skala, D. Davidovic, E. Afgan, I. Sovic, and Z. Sojat, "Scalable distributed computing hierarchy: Cloud, fog and dew computing," *Open Journal of Cloud Computing (OJCC)*, vol. 2, no. 1, pp. 16–24, 2015.

[3] S. Teerapittayanon, B. McDanel, and H. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *International Conference on Pattern Recognition*, 2016.

[4] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Advances in Neural Information Processing Systems*, 2015, pp. 3123–3131.

[5] B. McDanel, S. Teerapittayanon, and H. Kung, "Embedded binarized neural networks," in *International Conference on Embedded Wireless Systems and Networks*, 2017.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *arXiv preprint arXiv:1512.03385*, 2015.

[11] S. Yi, Z. Hao, Z. Qin, and Q. Li, "Fog computing: Platform and applications," in *Hot Topics in Web Systems and Technologies (HotWeb), 2015 Third IEEE Workshop on.* IEEE, 2015, pp. 73–78.

[12] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," *arXiv preprint arXiv:1603.05279*, 2016.

[13] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, "Spatially adaptive computation time for residual networks," *arXiv preprint arXiv:1612.02297*, 2016.

为了实现水平扩展的目标，我们提供了一种原则性的方法，通过本地和云聚合器的特征池来联合训练来自多个设备的输入的 DNN，并证明通过聚合来自每个设备的特征，我们可以显着提高系统在本地和云级别的准确性。每个设备上的过滤器都会自动调整，以处理地理上独特的输入，并共同努力实现相同的总体目标，从而实现较高的总体精度。此外，我们还表明 DDNN 提供跨终端设备的内置容错能力，并且在存在故障设备的情况下仍然能够实现高精度。

六．结论

在本文中，我们提出了一种新颖的分布式深度神经网络架构（DDNN），该架构分布在由云、边缘和终端设备组成的计算层次结构中。我们针对多视图、多摄像头数据集演示了 DDNN 从终端设备上的几个 NN 层或边缘垂直扩展到云中的许多 NN 层，并在多个终端设备上水平扩展。从不同设备传递的信息的聚合被内置到 DDNN 的联合训练中，并在推理时自动处理。这种方法简化了分布式云卸载的实施和部署，并实现了传感器融合和系统容错的自动化。

实验结果表明，使用我们的 DDNN 框架，经过适当训练的单个 DNN 可以映射到分布式计算层次结构，以满足目标应用程序的准确性、通信和延迟要求，同时获得与分布式计算相关的固有好处，例如容错和隐私。

与标准的云卸载方法相比，DDNN 通过在本地聚合器中退出许多样本并在需要额外处理时将紧凑的二进制特征表示发送到云来减少所需的通信。对于我们的评估数据集，与将原始传感器输入卸载到云中执行所有推理计算的 DNN 相比，DDNN 的通信成本降低了 20 倍以上。

DDNN 为进一步研究将 DNN 映射到分布式计算层次结构提供了一个框架。对于未来的工作，我们将研究 DDNN 在具有更大数据集、多种类型输入模式 [21] 和更多终端设备的应用程序上的性能。目前，DDNN 中的所有层都是二进制的。虽然由于设备上的空间有限，二进制层是终端设备的要求，但在云中则不需要。我们将探索其他类型的聚合方案和混合精度方案，其中终端设备使用二进制 NN 层，云使用混合精度或浮点 NN 层。

致谢

这项工作部分得到英特尔公司的捐赠，部分得到海军研究生院协议 N00244-15-0050 和 N00244-16-1-0018 下的海军供应系统司令部奖项的支持。

参考

[1] W. Shi，J. Cao，Q. Zhang，Y. Li，and L. Xu，“边缘计算：愿景与挑战”，*IEEE Internet of Things Journal*，卷。3、没有。5，第 637–646 页，2016 年。[2] K. Skala、D. Davidovic、E. Afgan、I. Sovic 和 Z. Sojat，“可扩展的分布式计算层次结构：云、雾和露计算”，*Open Journal of Cloud Computing (OJCC)*，卷。2、没有。1，第 16-24 页，2015 年。[3] S. Teerapittayanon、B. McDanel 和 H. Kung，“Branchynet：通过早期退出深度神经网络进行快速推理”，*International Conference on Pattern Recognition*，2016 年。[4] M. Courbariaux、Y. Bengio 和 J.-P。David，“Binaryconnect：在传播过程中使用二进制权重训练深度神经网络”，*Advances in Neural Information Processing Systems*，2015 年，第 3123-3131 页。[5] B. McDanel、S. Teerapittayanon 和 H. Kung，“嵌入式二值化神经网络”，*International Conference on Embedded Wireless Systems and Networks*，2017 年。[6] Y. LeCun、L. Bottou、Y. Bengio 和 P. Haffner，“基于梯度的学习应用于文档识别”，*Proceedings of the IEEE*，卷。86，没有。11，第 2278–2324 页，1998 年。[7] A. Krizhevsky、I. Sutskever 和 G. E. Hinton，“深度卷积神经网络的 Imagenet 分类”，*Advances in neural information processing systems*，2012 年，第 1097–1105 页。[8] K. Simonyan 和 A. Zisserman，“用于大规模图像识别的非常深的卷积网络”，*arXiv preprint arXiv:1409.1556*，2014 年。[9] C. Szegedy、W. Liu、Y. Jia、P. Sermanet、S. Reed、D. Anguelov、D. Erhan、V. Vanhoucke 和 A. Rabinovich，“深入卷积”，参见 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*，2015 年，第 1-9 页。

[10] K. He、X. Zhang、S. Ren 和 J. Sun，“图像识别的深度残差学习”，*arXiv preprint arXiv:1512.03385*，2015 年。

[11] S. Yi、Z.hao、Z.Qin 和 Q.Li，“雾计算：平台和应用”，*Hot Topics in Web Systems and Technologies (HotWeb), 2015 Third IEEE Workshop on*。IEEE，2015 年，第 73-78 页。

[12] M. Rastegari、V. Ordonez、J. Redmon 和 A. Farhadi，“Xnornet：使用二元卷积神经网络进行 Imagenet 分类”，*arXiv preprint arXiv:1603.05279*，2016 年。

[13] M.Figurnov、M.D.Collins、Y.Zhu、L.Zhang、J.Huang、D.Vetrov 和 R.Salakhutdinov，“残差网络的空间自适应计算时间”，*arXiv preprint arXiv:1612.02297*，2016 年。

[14] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, A. Senior, P. Tucker, K. Yang, Q. V. Le *et al.*, "Large scale distributed deep networks," in *Advances in neural information processing systems*, 2012, pp. 1223–1231.

[15] F. N. Iandola, K. Ashraf, M. W. Moskewicz, and K. Keutzer, "Firecaffe: near-linear acceleration of deep neural network training on compute clusters," *arXiv preprint arXiv:1511.00175*, 2015.

[16] J. Dean, "Large scale deep learning," in *Keynote GPU Technical Conference*, vol. 3, 2015, p. 2015.

[17] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[18] G. Roig, X. Boix, H. B. Shitrit, and P. Fua, "Conditional random fields for multi-camera object detection," in *2011 International Conference on Computer Vision*. IEEE, 2011, pp. 563–570.

[19] X. Li, W. Hu, C. Shen, Z. Zhang, A. Dick, and A. V. D. Hengel, "A survey of appearance models in visual object tracking," *ACM transactions on Intelligent Systems and Technology (TIST)*, vol. 4, no. 4, p. 58, 2013.

[20] B. McDanel, "Multiview multicamera dataset," https://www.dropbox.com/s/uk8c6iymy8nprc0/MVMC.npz, 2016, accessed: 2016-12-10.

[21] M. Cha, Y. Gwon, and H. Kung, "Multimodal sparse representation learning and applications," *arXiv preprint arXiv:1511.06238*, 2015.

[14] J. Dean、G. Corrado、R. Monga、K. Chen、M. Devin、M. Mao、A. Senior、P. Tucker、K. Yang、Q. V. Le *et al.*，“大规模分布式深度网络”，*Advances in neural information processing systems*，2012 年，第 1223-1231 页。

[15] F. N. Iandola、K. Ashraf、M. W. Moskewicz 和 K. Keutzer，“Firecaffe：计算集群上深度神经网络训练的近线性加速”，*arXiv preprint arXiv:1511.00175*，2015 年。

[16] J. Dean，“大规模深度学习”，*Keynote GPU Technical Conference*，卷。2015 年 3 月，第 3 页。2015年。

[17] D. Kingma 和 J. Ba，“Adam：一种随机优化方法”，*arXiv preprint arXiv:1412.6980*，2014。[18] G. Roig、X. Boix、H. B. Shitrit 和 P. Fua，“多相机目标检测的条件随机场”，*2011 International Conference on Computer Vision*。IEEE，2011 年，第 563–570 页。[19] X. Li，W. Hu，C. Shen，Z. Zhang，A. Dick，A. V. D. Hengel，“视觉对象跟踪中外观模型的调查”，*ACM transactions on Intelligent Systems and Technology (TIST)*，卷。4、没有。4，p。58, 2013。[20] B. McDanel，“多视图多摄像头数据集”，https://www.dropbox.com/s/uk8c6iymy8nprc0/MVMC.npz，2016 年，访问时间：2016-12-10。[21] M. Cha、Y. Gwon 和 H. Kung，“多模态稀疏表示学习和应用”，*arXiv preprint arXiv:1511.06238*，2015 年。